

基于差分进化改良遗传算法的投弹策略优化

摘要

在目标防护场景中，可利用无人机投放烟幕，挡住导弹对真实目标的视线。在烟幕有效时间及有效范围内能够生效；无人机受速度、航向、投放和起爆时序以及起爆高度的限制；导弹有固定来向和命中时刻，其中是否达成有效遮蔽，取决于导弹到目标采样点的视线是否落在烟幕作用范围内。无人机的投弹策略显得尤为重要，本文针对烟幕干扰弹投放问题，结合三维坐标系下物体运动模型与优化模型，提出一套有效的烟幕弹投放策略。

针对问题一，本文首先建立了相关物体的**运动模型**，对于**遮挡判定模型**，先将导弹假想为一点光源，构建了由真目标圆柱体几何中心的粗略判定，到**边界离散化**获取采样点的**二步式解析几何判定**方法，采样点的选取数量依据**随机实验与几何概型**确定，在保证结果准确率的情况下有效减少判定所用时间，最后通过**数值模拟及时间的差分离散化**，求解得到有效遮蔽时间为**1.391s**。

针对问题二，首先建立了由无人机航向，航速，投弹时间，引爆时间为决策变量，最大化有效遮蔽时间为目标的优化模型。然后采用**遗传算法**求解该问题，为了降低求解过程的复杂度以及防止陷入局部最优，在此基础上嵌入**内部剪枝解码**，逐步缩小搜索范围，更快得到遗传算法所对应的优秀“基因”，在实际运行上，降低了求解的时间复杂度，最后得出最大有效遮蔽时间为**4.58s**，相对于第二问有较大提升，证明了优化的有效性。

针对问题三，首先建立了相对于问题二增设了两枚烟幕弹的投放时间以及引爆时间为决策变量，最大化**多弹遮蔽时间窗口并集**为目标的优化模型，考虑到相对于第二问增设多变量的情况，选择采用**差分进化算法结合遗传算法**进行求解，进一步防止陷入局部最优，并且考虑到单颗烟幕弹不满足有效遮蔽，多弹协同却能够达成有效遮蔽的情况，增加了达成有效遮蔽的情况，最后单独对**时间窗口**进行优化以减少重叠时间，得到遮蔽时间并集最优为**6.46s**，并将相关参数保存在 result1 中。

针对问题四，面向导弹 M1 命中前三机**烟幕遮蔽并集最大化**，期望三机能够**接力遮蔽**，引入分层优化思想，**粗化层**：基于第二问的求解方式下，分别求解 FLY2、FLY3 单机最优与其遮蔽时间段，并将 FLY1 既定方案与二者做时间并集，得到**参考基准与精细优化的初始范围**。**精化层**：固定 FLY1，以单机最优为**邻域初值**，仅对 FLY2、FLY3 的速度、方向与时序做小扰动搜索，降低求解复杂度，沿用第三问的 **DEGA**（差分进化+遗传）算法进行求解，最后求得三机烟幕遮蔽最优时间为**11.59s**，并将相关参数保存在 result2 中。

针对问题五，首先明确导弹遮蔽时间独立计算原则，采用**分组策略及分层优化思想**，分组策略旨在对无人机进行任务分配，基于各无人机与导弹相对空间位置的特殊性，构造了相关**适配度矩阵**，为每架无人机指派干扰的导弹主次要目标，并保证每枚导弹至少有两架无人机覆盖，采用**差分进化（DE）**算法进行子任务优化，对三枚导弹的独立拦截任务分别进行运行求解，将子任务最优解汇总形成总体最优解，最后求得遮蔽总遮蔽时间为 **21.29s**，并将相关参数保存在 result3 中。

关键词：烟幕弹投放策略优化 遮挡判定 DEGA 算法 分层优化 多机多弹协同

一、问题重述

1.1 问题背景

在现代战争中，烟幕干扰弹作为电子对抗与光电对抗的关键装备，通过化学燃烧或爆炸分散形成烟幕或气溶胶云团，可有效遮蔽己方目标、干扰敌方探测系统，是保护重要目标的重要战术手段。随着技术迭代，烟幕干扰弹已实现定点精确抛撒，抛撒前可精准控制其抵达预设位置，再通过时间引信时序控制起爆时间，大幅提升了干扰的时效性与针对性。而无人机凭借机动性强、成本较低、可精确定位投放等优势，成为烟幕干扰弹投放的理想平台，能在复杂战场环境中灵活调整航迹，实现对来袭导弹的高效拦截。

1.2 问题的提出

本问题聚焦无人机投放烟幕干扰弹干扰来袭空地导弹的场景。每架无人机投放两枚干扰弹间隔至少 1s。干扰弹脱离后受重力作用，起爆形成球状烟幕云团，以 3m/s 匀速下沉，云团中心 10m 内 20s 可为目标有效遮蔽。来袭空地导弹速度 300m/s，目标为掩护真目标设的假目标。以假目标为原点建系，真目标下底面圆心(0,200,0)。警戒雷达发现时，3 枚导弹 M1(20000,0,2000)、M2(19000,600,2100)、M3(18000,-600,1900)；5 架无人机 FY1(17800,0,1800)、FY2(12000,1400,1400)、FY3(6000,-3000,700)、FY4(11000,2000,1800)、FY5(13000,-2000,1300)。控制中心发现目标立即指派任务，无人机领命可瞬时调向，以 70~140m/s 等高度匀速飞行，航向、速度确定后一直不变。核心目标是设计无人机投放策略，通过优化飞行参数与投放时序，最大化真目标的有效遮蔽时间^[2]。

通过建立模型，解决以下问题：

- (1) 给定 FY1 飞行参数（120m/s 朝假目标飞行，1.5s 投放、3.6s 起爆），计算单弹对 M1 的有效遮蔽时长；
- (2) 优化 FY1 单弹投放策略（飞行方向、速度、投放点、起爆点），最大化遮蔽时间；
- (3) 设计 FY1 三弹投放策略（满足间隔约束），实现对 M1 的连续遮蔽；
- (4) 协调 FY1、FY2、FY3 各投 1 枚弹，协同干扰 M1；
- (5) 用 5 架无人机（每架至多投 3 枚）干扰 M1、M2、M3，输出全局最优投放策略并保存结果。

二、模型假设与符号说明

- (1) 忽略空气阻力的影响，研究对象可以保持匀速，取重力加速度为 $9.8m/s^2$ 。
- (2) 烟幕云团在有效时间、一定范围内遮蔽性能保持恒定，忽略密度衰减和扩散稀释效应。同时可以简化为球形区域，忽略形状不规则性。
- (3) 导弹的形状、大小对研究的问题影响可忽略不计，可以视为质点，且导弹的爆炸是瞬时完成的，不存在任何延时。

表2.1 符号说明

符号	说明	单位
t	运动时间	s
V	运动速度	m/s
$\vec{u}$	速度方向单位向量	/
d_j	烟幕云团球的球心点 $C_j(t)$ 到空间直线 M_1T 的距离	m
O	假目标的位置坐标	/
e_i	i 方向单位向量	/
g	重力加速度常数	m/s^2
T	真目标的位置坐标	/
Δt	时间步长	s
$P_{i,0}$	第 i 架飞机在初始时刻的位置坐标	/
$M_{k,0}$	第 k 枚导弹在初始时刻的位置坐标	/
$P_i(t)$	第 i 架飞机在 t 时刻的位置坐标	/
$t_{rel,j}$	第 j 枚干扰弹投放时刻	s
$t_{burst,j}$	第 j 枚干扰弹起爆时刻	s
$\Delta t_{burst,j}$	第 j 枚干扰弹起爆延时	s
$P_{rel,j}(t)$	第 j 枚干扰弹投放位置坐标	/
$P_{burst,j}(t)$	第 j 枚干扰弹起爆位置坐标	/
$C_j(t)$	第 j 个烟幕云团中心在 t 时刻的位置坐标	/
$M_k(t)$	第 k 枚导弹在 t 时刻的位置坐标	/
$T_{covered}$	总的有效遮蔽时长	s
$f_jCovered$	第 j 个烟幕云团的遮蔽指示函数	/

三、问题分析

3.1 问题一的分析

针对问题一单枚烟幕弹对指定导弹的有效遮蔽时长，其本质是通过建立多体运动模型和空间几何关系，量化烟幕云团与导弹-真目标连线的时空交集。首先需明确各运动体的位置参数，在此基础上，遮蔽有效性的判定需围绕真目标表面展开——真目标为圆柱体结构，其外表面边界是需保护的关键区域，因此遮蔽有效性的判定为导弹与真目标表面任意点的连线是否被烟幕云团遮挡，即云团中心到该连线的距离是否小于云团半径。判定分三步构建数学关系：通过无人机运动参数计算烟幕弹投放点，结合起爆时间和下沉速度得到云团中心在任意时刻的三维坐标；基于导弹初始位置和速度推导其轨迹方程，确定任意时刻的导弹位置；针对真目标圆柱体表面的离散采样点，建立导弹到各采样点的空间直线方程，利用向量叉积公式计算烟幕云团中心到每条直线的距离，当距离小于 10 米且云团处于有效持续期时，满足时间空间条件，判定为遮蔽状态。时间维度上，将离散时间点的遮蔽状态转化为连续时间的总遮蔽时，通过二值指示函数标记每个时刻是否满足遮蔽条件，最终以时间步长乘以遮蔽状态为 1 的步数，得到总有效遮蔽时长。

3.2 问题二的分析

针对问题 2“单无人机单弹优化”，此时需将航向角，速度，投弹时间，起爆时间作为决策变量，先构建无人机的运动轨迹方程，进而推导出烟幕弹起爆点位置与决策变量的关系，最终建立有效遮蔽时间关于航向角和速度的函数表达式并以此作为目标函数，并在各个约束的作用下通过优化算法求解最优参数。考虑到题目中各问题是层层递进式的，本文选用遗传算法进行模型求解，因其易与分层优化、局部搜索、枚举剪枝、二分判定等组合形成模因算法（memetic GA），提升收敛速度与解质量。在求解过程中，本文嵌入剪枝解码，逐步缩小搜索范围进行求解。

3.3 问题三的分析

针对问题 3“单无人机三弹优化”，在问题二的基础上增加了两颗投弹，此时的目标是使得三弹遮蔽时间窗口并集最大化，则需要在单弹优化的基础上增加时间协调思想，即通过调整三枚烟幕弹的投放时间和起爆时间，在满足 1 秒投放间隔约束的前提下，使多弹形成的遮蔽区域在时间上连续、空间上覆盖导弹轨迹，在遮蔽效果上产生接力，从而最大化总遮蔽时长。此时需增设多弹投放时间和起爆延迟作为决策变量，构建包含时间间隔约束的多目标优化模型，考虑在遗传算法的基础上引入差分进化算法，以加快收敛速度并且减小陷入局部最优解的情况。

3.4 问题四的分析

问题 4 要求多架无人机协同投放烟幕弹对单枚指定导弹实现最大化有效遮蔽，相当于第二问增加了两架无人机的变量。需基于各无人机的初始位置，与导弹轨迹的空间距离的关系进行任务分配，同样是接力式的思想，初始位置更接近导弹轨迹的无人机优先参与，速度较快的无人机可承担多弹投放任务，采用分层优化的思想，先沿用第二问求解方式，修改初始参数得到 FLY2 FLY3 在单机单弹的情景下的最优情况，并以得到的相关参数为范本，进行下一步精细优化，依然采用基于差分进化算法改进的遗传算法进行求解，最后得到最大时间并集，绘制每颗烟幕弹达成遮蔽的时间区间。

3.5 问题五的分析

问题 5 是问题三四的结合扩展，五架无人机对三枚导弹的协同干扰，每架无人机最多投放三枚烟幕弹的情景下，空间复杂度以及时间复杂度均上升到了常规优化算法无法求解的情况，考虑先对问题进行拆解，简化，由于各导弹及无人机空间位置的特殊性，本文采用分组拦截策略与分层优化思想，粗化层将每架无人机与其最易可拦截目标进行配对，进行大方向上的粗略搜索，降低求解的复杂度，精细层采用寻优搜索的方式获取，确保求解结果的正确性，内核依然采用 优化算法，最后得出了理的烟幕干扰弹投放策略。

四、模型的建立与求解

4.1 问题一的模型建立与求解

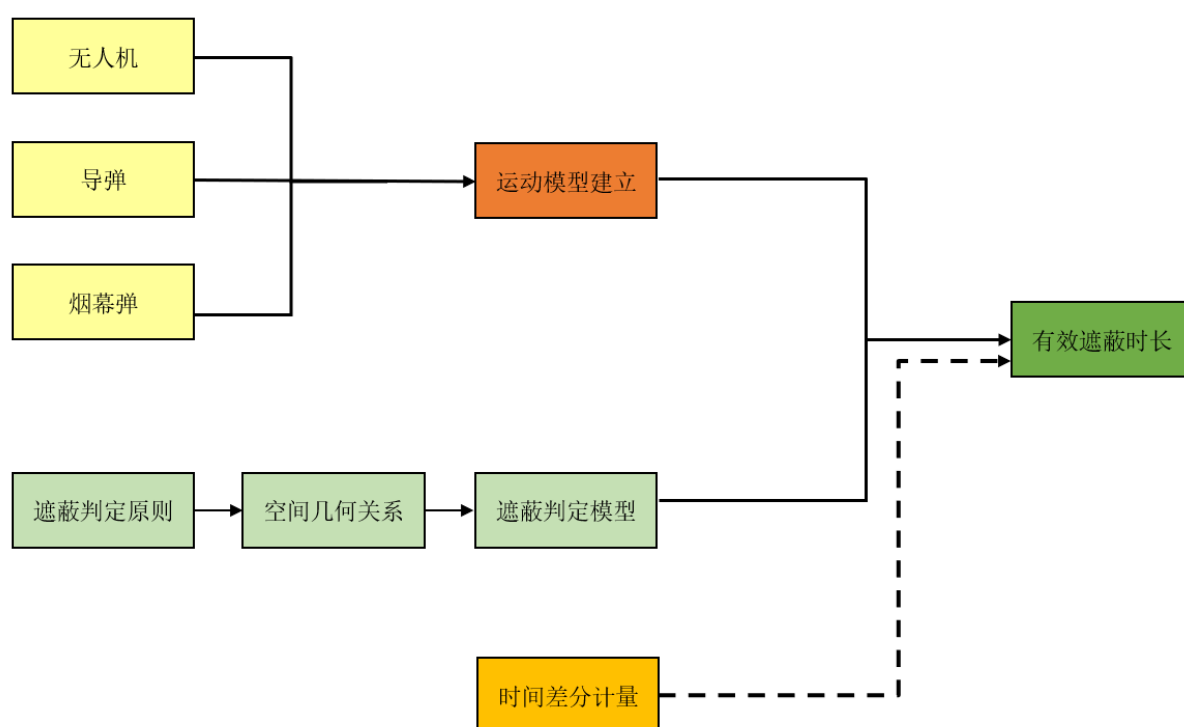


图4.1 问题一思路流程图

4.1.2 物体运动模型的建立

以假目标位置为原点 O ，建立 xyz 立体平面。

4.1.1.1 导弹 M1 的运动模型

导弹 M1 的运动方式为匀速直线运动。在该模型中，核心假设导弹在飞行过程中不受任何外力干扰,如重力、空气阻力或控制指令等，其速度矢量保持恒定，运动轨迹为一条直线。

导弹 M1 初始位置：

$$M_{1,0} = (20000, 0, 2000)$$

飞行速度大小为 300m/s ，飞行方向指向假目标原点 O 。从导弹初始位置指向原点的单位向量可表示为 $-e_x$ ，其中负号表示方向指向坐标原点，则导弹飞行速度向量可以表示为

$$\vec{u}_m = \frac{O - M_{1,0}}{\|O - M_{1,0}\|} = -300e_x \quad (4.1)$$

任意时刻导弹 M1 运动方程可表示为:

$$M_1(t) = M_{1,0} + \vec{u}_m \cdot t = (20000 - 300t, 0, 2000) \quad (4.2)$$

4.1.1.2 无人机 FY1 的运动模型

无人机 FY1 的运动采用等高度匀速直线运动模型描述,其核心假设为飞行过程中保持固定高度、以恒定速度沿直线飞行,忽略空气阻力、风力干扰及机动变化等因素。根据题目约束, FY1 受领任务后可瞬时调整飞行方向,随后航向与速度一旦确定即保持不变,仅通过飞行方向(水平角度)与速度的优化,控制烟幕弹投放点。

FY1 初始位置:

$$P_{1,0} = (17800, 0, 1800)$$

FY1 飞行的水平高度不变:

$$u_z = 0$$

FY1 飞行的单位方向向量:

$$\|\vec{u}\| = \sqrt{u_x^2 + u_y^2 + u_z^2} = 1, \vec{u} = (\cos\theta, \sin\theta, 0) \quad (4.3)$$

FY1 飞行速度约束:

$$70 \leq v \leq 140$$

FY1 投放时刻位置:

$$P_1(t) = P_{1,0} + vt_{rel}\vec{u} = (17800 + vt_{rel}\cos\theta, vt_{rel}\sin\theta, 1800) \quad (4.4)$$

4.1.1.3 烟幕弹投放与起爆位置模型

烟幕弹的投放与起爆位置模型基于无人机 FY1 的等高度匀速直线运动构建: FY1 受领任务后,沿调整后的方向以恒定速度飞行, t_{rel} 时刻投放烟幕弹,投放点位置为初始位置加水平方向运动距离;烟幕弹扔出后, xOy 平面方向与飞机速度方向一致,竖直方向做自由落体运动;爆开后,在竖直方向匀速下落, xOy 平面方向速度归零。烟幕弹整体做平抛运动,直至 t_{burst} 时刻起爆。投放时间:

$$t_{rel}$$

起爆时间:

$$t_{burst} = t_{rel} + \Delta t_{burst} \quad (4.5)$$

此时起爆点位置方程可描述为:

$$\begin{aligned} P_{burst} &= P_{1,0} + vt_{burst}\vec{u} - 0.5g\Delta t_{burst}^2 \cdot e_z \\ &= (17800 + vt_{burst}\cos\theta, vt_{burst}\sin\theta, 1800 - 4.9 \times (t_{burst} - t_{rel})^2) \end{aligned} \quad (4.6)$$

起爆后,烟幕云团以 3m/s 匀速下沉,水平坐标保持起爆点的水平位置。起爆后的烟幕云团球心点 P 任意时刻位置可表示为:

$$C_j(t) = (17800 + vtc\cos\theta, vts\sin\theta, 1800 - 4.9 \times t^2 - 3 \times (t - t_{burst})) \quad (4.7)$$

4.1.3 遮蔽判定模型的建立

4.1.2.1 遮蔽判定原则的确立

在确立遮蔽判定模型之前,需要明确何种情况下才算“遮挡”,其大致机理可表述为:云团内均匀分布的悬浮颗粒,对导弹制导系统常用的可见光、近红外或中红外波段电磁波产生强烈的散射与吸收效应,导致导弹无法“发现”真目标^[1]。也就是把导弹想象成一质点,且向各个方向散发光线的点光源,而光线进入烟幕有效范围时被反射吸收,

无法继续前进，当去往真目标的所有光线均被吸收时，此时认为烟幕对真目标是有效遮挡^[2]。

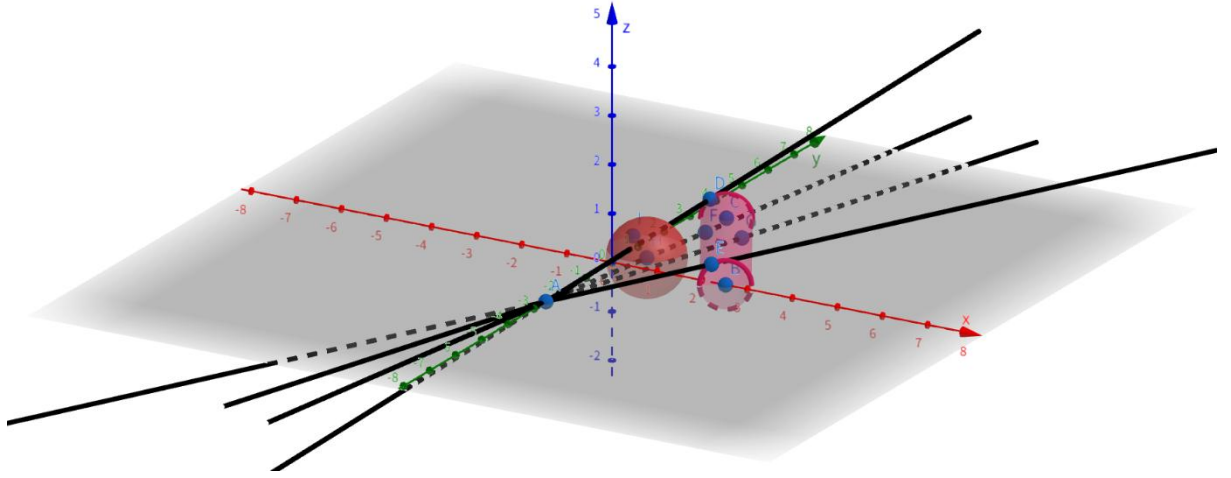


图4.2 遮蔽判定原则示意图

4.1.2.2 有效遮蔽判定标准的确立

空间几何条件：烟幕球的球心到导弹与真目标上任意一点 T 的连线的距离小于烟幕球的有效半径，导弹无法直“见”真目标。

真目标圆柱体外表面边界上点的方程，即圆柱面方程：

$$\begin{cases} x^2 + (y - 200)^2 = 49 \\ 0 \leq z \leq 10 \end{cases} \quad (4.8)$$

外表面边界上点 T ：

$$T = (7\cos\theta, 200 + 7\sin\theta, z), \theta \in [0, 2\pi), z \in [0, 10]$$

导弹与真目标上任意一点的连线的方向矢量：

$$\vec{s} = (l, m, n)$$

烟幕球的空间方程：

$$(x - x_0)^2 + (y - y_0)^2 + (z - z_0)^2 \leq r^2 \quad (4.9)$$

空间直线 M_1T 方程：

$$\frac{x - x_1}{l} = \frac{y - y_1}{m} = \frac{z - z_1}{n} \quad (4.10)$$

烟幕云团球的球心点 P 到空间直线 M_1T 的距离为：

$$d = \frac{\begin{vmatrix} i & j & k \\ x_1 - x_0 & y_1 - y_0 & z_1 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} \quad (4.11)$$

当 d 小于烟幕球的有效半径且大于0时认为满足空间判定，即：

$$0 < d < 10$$

时间序列条件：云团形成后的有效遮蔽时间 ΔT 小于有效持续时间：

$$0 \leq \Delta T < 20s$$

4.1.2.3 真目标的边界离散化

对圆柱体进行水平切割，取 n_s 个圆面（包含上下底面），同时对任一圆面边界等分取 n_θ 个边界点 T 。由于样本圆柱体有无数个点，圆柱体是几何空间中的有限区域，同时样本点位于外表面的概率与圆柱体的位置形状无关，所以 n_s 和 n_θ 的取值可由随机实验方法

以及几何概型确定。

为了简化计算，首先进行一个初步的判定，即导弹与真目标几何中心的连线是否通过烟幕球。然后进行更严格的判定，如果 $n_s \cdot n_\theta$ 个边界点 T 与 M_1 所形成直线均满足约束条件，即可判定遮蔽有效。

确认真目标圆柱侧面采样规模过程可视化如下图所示(烟幕球部分遮挡情况下，采样点的遮挡率随取样规模(密度)的变化曲线)

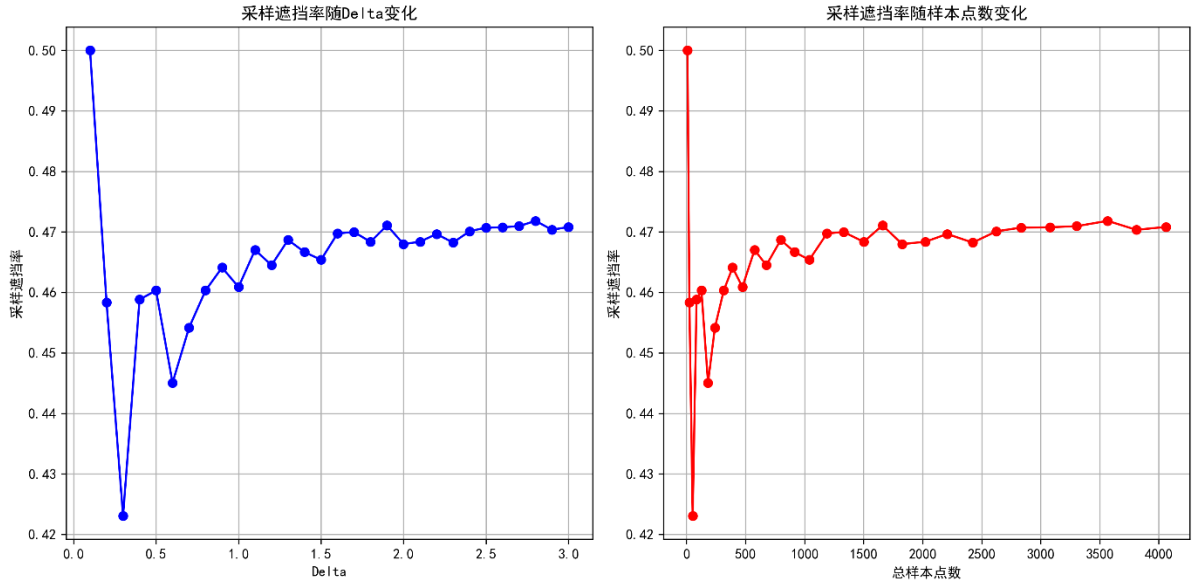


图4.3 采样点的遮挡率变化曲线图

4.1.4 遮蔽时间的计算

通过积分形式的目标函数量化烟幕弹对导弹的有效遮蔽时长。引入时间步长、指示函数和积分运算，将离散时间点的遮蔽状态转化为连续时间的总遮蔽时长。

目标函数:对烟幕弹有效持续期（20s）内的所有时间步长求和，最大化积分结果。

$$T_{covered} = \int_{t_{start}}^{t_{start}+20} f_{Covered}(t) dt \quad (4.12)$$

其中 t_i 表示第 i 个时间步长的时刻， $t_i = t_{burst} + \Delta t \cdot i$ 。 τ 为引爆时间， t 为投放烟幕弹时间， T 为广义时刻， $T_{covered}$ 为有效遮蔽时长。

二值遮蔽状态指示函数 $f_{Covered}$ ，用于判断某时刻是否满足 t_i 遮蔽条件。

$$f_{Covered} = \begin{cases} 0, & \text{非遮蔽时刻} \\ 1, & t_i \text{时刻遮蔽} \end{cases} \quad (4.13)$$

4.1.5 模型的求解

模型的求解采用 python 进行数值模拟的方法，考虑到时间的连续性不利于求和，定义一时间步长 Δt 取 0.001s，将时间域离散化，则时间积分可离散为：

$$\sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt$$

t_i 表示第 i 个时间步长的时刻， $t_i = t_{burst} + \Delta t \cdot i$ 。 τ 为引爆时间， t 为投放烟幕弹时间， T 为广义时刻。

求解过程如下流程图所示：

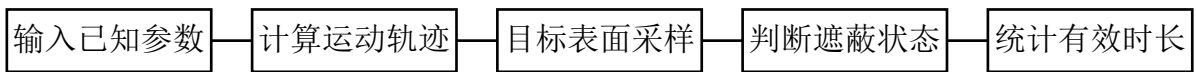


图4.4

最终得到有效遮蔽时长 $\Delta T = 1.39s$

求解可视化如下图所示：

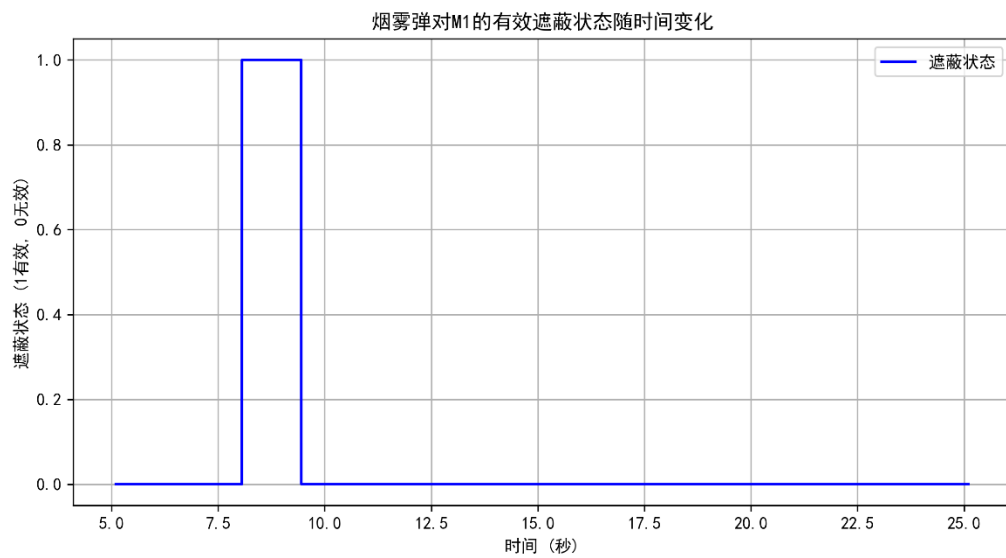


图4.5 求解可视化图

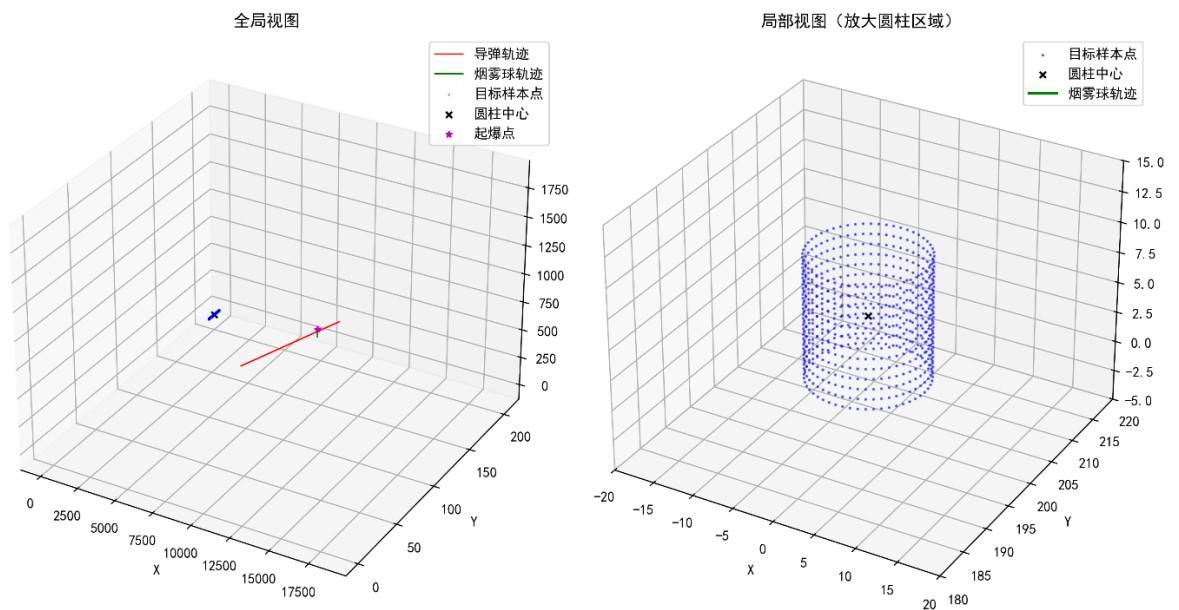


图4.6 空间示意图

问题一求解结果

表4.1 问题一的结果

参数	数值
总样本点数	676
起爆点位置	(178188, 0, 1736.496)
有效遮蔽开始时刻	8.0570s
有效遮蔽结束时刻	9.4480s
有效遮蔽总时长	1.3910s

4.2 问题二的模型建立与求解

4.2.1 运动模型的建立

4.2.2.1 导弹 M1 的运动模型

导弹 M1 的运动方式为匀速直线运动。在该模型中，核心假设导弹在飞行过程中不受任何外力干扰,如重力、空气阻力或控制指令等，其速度矢量保持恒定，运动轨迹为一条直线。

导弹 M1 初始位置：

$$M_{1,0} = (20000, 0, 2000)$$

飞行速度大小为 300m/s，飞行方向指向假目标原点 O 。从导弹初始位置指向原点的单位向量可表示为 $-e_x$ ，其中负号表示方向指向坐标原点，则导弹飞行速度向量可以表示为

$$\vec{u}_m = \frac{O - M_{1,0}}{\|O - M_{1,0}\|} = -300e_x \quad (4.14)$$

任意时刻导弹 M1 运动方程可表示为：

$$M_1(t) = M_{1,0} + \vec{u}_m \cdot t = (20000 - 300t, 0, 2000) \quad (4.15)$$

4.2.1.2 无人机 FY1 的运动模型

无人机 FY1 的运动采用等高度匀速直线运动模型描述，其核心假设为飞行过程中保持固定高度、以恒定速度沿直线飞行，忽略空气阻力、风力干扰及机动变化等因素。根据题目约束，FY1 受领任务后可瞬时调整飞行方向，随后航向与速度一旦确定即保持不变，仅通过飞行方向（水平角度）与速度的优化，控制烟幕弹投放点。

FY1 初始位置：

$$P_{1,0} = (17800, 0, 1800)$$

FY1 飞行的水平高度不变：

$$u_z = 0$$

FY1 飞行的单位方向向量：

$$\|\vec{u}\| = \sqrt{u_x^2 + u_y^2 + u_z^2} = 1, \vec{u} = (\cos\theta, \sin\theta, 0) \quad (4.16)$$

FY1 飞行速度约束：

$$70 \leq v \leq 140$$

FY1 投放时刻位置：

$$P_1(t) = P_{1,0} + vt_{rel}\vec{u} = (17800 + vt_{rel}\cos\theta, vt_{rel}\sin\theta, 1800) \quad (4.17)$$

4.2.1.3 烟幕弹投放与起爆位置模型

烟幕弹的投放与起爆位置模型基于无人机 FY1 的等高度匀速直线运动构建：FY1 受领任务后，沿调整后的方向以恒定速度飞行， t_{rel} 时刻投放烟幕弹，投放点位置为初始位

置加水平方向运动距离；烟幕弹扔出后， xOy 平面方向与飞机速度方向一致，竖直方向做自由落体运动；爆开后，在竖直方向匀速下落， xOy 平面方向速度归零。烟幕弹整体做平抛运动，直至 t_{burst} 时刻起爆。

投放时间：

$$t_{rel}$$

起爆时间：

$$t_{burst} = t_{rel} + \Delta t_{burst} \quad (4.18)$$

此时起爆点位置方程可描述为：

$$\begin{aligned} P_{burst} &= P_{1,0} + vt_{burst}\vec{u} - 0.5g\Delta t_{burst}^2 \cdot e_z \\ &= (17800 + vt_{burst}\cos\theta, vt_{burst}\sin\theta, 1800 - 4.9 \times (t_{burst} - t_{rel})^2) \end{aligned} \quad (4.19)$$

起爆后，烟幕云团以 3m/s 匀速下沉，水平坐标保持起爆点的水平位置。烟幕云团中心在 t 时刻的位置坐标可表示为：

$$C_j(t) = (17800 + v\cos\theta, v\sin\theta, 1800 - 4.9 \times \Delta t_{burst}^2 - 3 \times (t - t_{burst})) \quad (4.20)$$

4.2.2 有效遮蔽时长优化模型

4.2.2.1 目标函数：

目标是使得干扰弹对导弹 M1 的有效遮蔽时长 ΔT 尽可能长，可表示为：

$$Fit = \max \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt \quad (4.21)$$

其中： Δt 为时间步长， t_i 表示第 i 个时间步长的时刻， $t_i = t_{burst} + \Delta t \cdot i$ 。 τ 为引爆时间， t 为投放烟幕弹时间， T 为广义时刻， ΔT 为有效遮蔽时长。

4.2.2.2 决策变量：

FY1 的飞行方向角度 θ ，无人机在 xOy 平面上的飞行方向与 x 轴正方向的夹角，飞行方向的单位向量：

$$\vec{u} = (\cos\theta, \sin\theta, 0)$$

FY1 飞行速度 v ：满足条件：

$$70 \leq v \leq 140$$

烟幕干扰弹投放时间 t_{rel} ：无人机 FY1 受领任务后到投放干扰弹经历的时间，满足条件：

$$t_{rel} \geq 0$$

烟幕干扰弹起爆时间 t_{burst} ：无人机 FY1 受领任务后到投放干扰弹起爆经历的时间，满足条件：

$$t_{burst} \geq t_{rel}$$

4.2.2.3 约束条件：

无人机速度大小约束：

$$70 \leq v \leq 140$$

投放时间非负约束：

$$t_{rel} \geq 0$$

起爆时间约束：

$$1800 - 4.9 \times (t_{burst} - t_{rel})^2 \geq 0$$

起爆时序先后约束：

$$t_{burst} \geq t_{rel}$$

飞行方向角约束:

$$\theta \in [0, 2\pi)$$

有效时间约束:

$$0 \leq \max \left\{ \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt < 20 \right. \quad (4.22)$$

烟幕云团球的球心点 P 到空间直线 M_1T 的距离小于球的半径

$$0 \leq d = \frac{\begin{vmatrix} i & j & k \\ x_1 - x_0 & y_1 - y_0 & z_1 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10 \quad (4.23)$$

优化模型总结:

$$\begin{aligned} & \max \left\{ \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt \right. \\ & s. t. \begin{cases} 70 \leq v \leq 140 \\ \theta \in [0, 2\pi) \\ t_{rel} \geq 0 \\ 1800 - 4.9 \times (t_{burst} - t_{rel})^2 \geq 0 \\ t_{burst} \geq t_{rel} \\ d = \frac{\begin{vmatrix} i & j & k \\ x_1 - x_0 & y_1 - y_0 & z_1 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10 \end{cases} \end{aligned} \quad (4.24)$$

4.2.3 遗传算法求解单目标优化问题

遗传算法 GA 是一种模拟生物进化过程的启发式优化算法，基于生物进化理论通过模拟种群的选择、交叉、变异等遗传操作，以搜索全局最优解^[2]。

遗传算法 GA 的基本操作:

(1) 初始化种群: 将问题的可行解转化为“染色体”，直接用决策变量值进行实数个体编码。同时随机生成 N 个个体，构成初始解集合。

(2) 适应度函数: 有效遮蔽时长 ΔT 越长，表明个体适应度越高，因此直接定义适应度函数为有效遮蔽时长 ΔT 。

(3) 遗传操作: 选择，算术交叉，变异

(4) 种群迭代: 将子代与父代合并，保留最优个体，形成新一代种群，重复“选择-交叉-变异”操作，直至满足适应度收敛。

具体遗传算法流程如图所示:

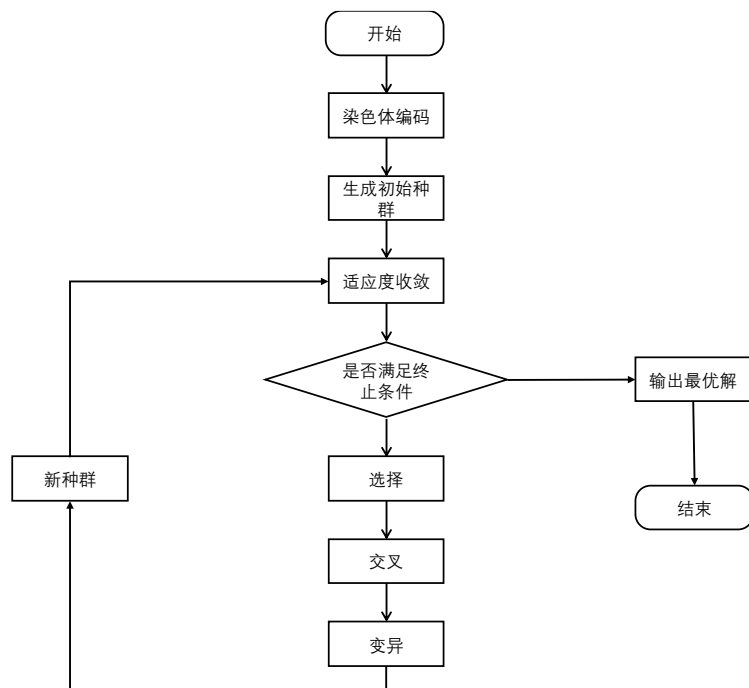


图4.7 遗传算法流程图

考虑到在这种多决策变量优化的情况下，遗传算法可能陷入局部最优解，提出在遗传算法做大规模搜索的基础上，内部嵌入剪枝解码器，剪枝思想可以通俗理解为裁剪去不可能通往正确答案的参数路径，可以理解为在运行中逐步缩小搜索范围，运行过程中逐步将飞行角度限定在接近 x 轴正向的一定范围内，由此降低了运行复杂度，最终得出最大有效遮蔽时长为 **4.58s**。相关可视化过程可如下图所示：

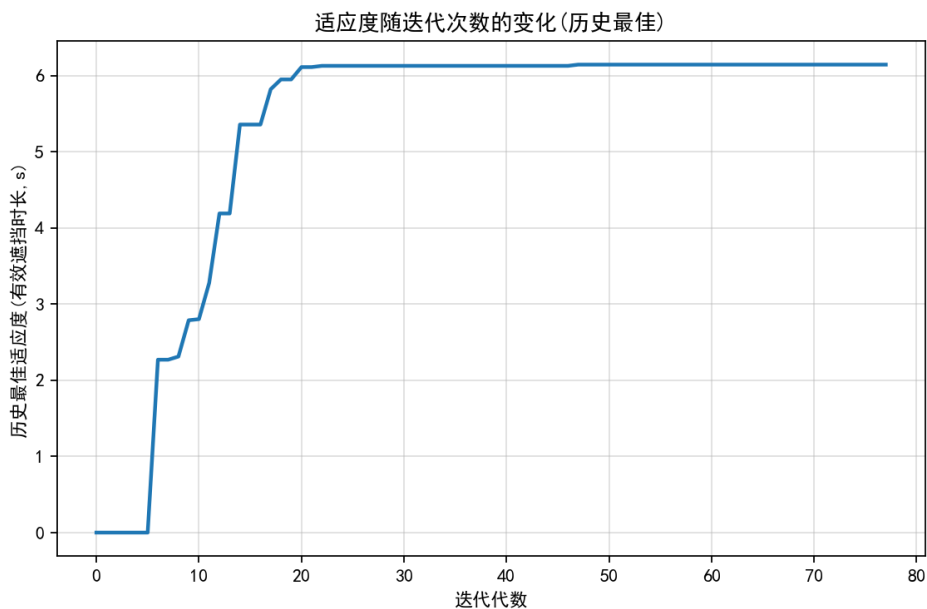


图4.8 FY1 适应度曲线图

表4.2 问题二的结果

参数	数值
相对 x 轴无人机运动方向角	5.4408°
无人机运动速度	111.6373m/s
投放延迟时间	0.5371s
起爆延迟时间	0.4851s
烟幕干扰弹投放点位置	(17859.686, 5.685, 1800)
烟幕干扰弹起爆点位置	(17913.593, 10.819, 1798.847)
有效遮蔽开始时刻	1.232s
有效遮蔽结束时刻	5.812s
有效遮蔽总时长	4.580s

4.3 问题三的模型建立与求解

4.3.1 运动模型的建立

4.3.1.1 导弹 M1 的运动模型

导弹 M1 的运动方式为匀速直线运动。在该模型中，核心假设导弹在飞行过程中不受任何外力干扰,如重力、空气阻力或控制指令等，其速度矢量保持恒定，运动轨迹为一条直线。

导弹 M1 初始位置：

$$M_{1,0} = (20000, 0, 2000)$$

飞行速度大小为 300m/s，飞行方向指向假目标原点 O 。从导弹初始位置指向原点的单位向量可表示为 $-e_x$ ，其中负号表示方向指向坐标原点，则导弹飞行速度向量可以表示为

$$\vec{u}_m = \frac{O - M_{1,0}}{\|O - M_{1,0}\|} = -300e_x \quad (4.25)$$

任意时刻导弹 M1 运动方程可表示为：

$$M_1(t) = M_{1,0} + \vec{u}_m \cdot t = (20000 - 300t, 0, 2000) \quad (4.26)$$

4.3.1.2 无人机 FY1 的运动模型

无人机 FY1 的运动采用等高度匀速直线运动模型描述，其核心假设为飞行过程中保持固定高度、以恒定速度沿直线飞行，忽略空气阻力、风力干扰及机动变化等因素。根据题目约束，FY1 受领任务后可瞬时调整飞行方向，随后航向与速度一旦确定即保持不变，仅通过飞行方向（水平角度）与速度的优化，控制烟幕弹投放点。

FY1 初始位置：

$$P_{1,0} = (17800, 0, 1800)$$

FY1 飞行的水平高度不变：

$$u_z = 0$$

FY1 飞行的单位方向向量：

$$\|\vec{u}\| = \sqrt{u_x^2 + u_y^2 + u_z^2} = 1, \vec{u} = (\cos\theta, \sin\theta, 0) \quad (4.27)$$

FY1 飞行速度约束：

$$70 \leq v \leq 140$$

FY1 投放第 j 枚烟幕弹时刻位置：

$$P_1(t) = P_{1,0} + vt_{rel,j}\vec{u} = (17800 + vt_{rel,j}\cos\theta, vt_{rel,j}\sin\theta, 1800) \quad (4.28)$$

4.3.1.3 烟幕弹投放与起爆位置模型

烟幕弹的投放与起爆位置模型基于无人机 FY1 的等高度匀速直线运动构建: FY1 受领任务后, 沿调整后的方向以恒定速度飞行, $t_{rel,j}$ 时刻投放第 i 枚烟幕弹, $t_{rel,j+1}$ 时刻投放第 $j+1$ 枚烟幕弹, 两者至少间隔 1 秒烟幕弹投放后。所有烟幕弹投放后, xOy 平面方向与飞机速度方向一致, 竖直方向做自由落体运动; 爆炸后, 在竖直方向匀速下落, xOy 平面方向速度归零。

第 j 枚烟幕弹投放时间:

$$t_{rel,j}$$

第 j 枚烟幕弹起爆时间:

$$t_{burst,j} = t_{rel,j} + \Delta t_{burst,j} \quad (4.29)$$

此时起爆点位置方程可描述为:

$$\begin{aligned} P_{burst,j} &= P_{1,0} + vt_{burst,j}\vec{u} - 0.5g\Delta t_{burst,j}^2 \cdot e_z \\ &= (17800 + vt_{burst,j}\cos\theta, vt_{burst,j}\sin\theta, 1800 - 4.9 \times (t_{burst,j} - t_{rel,j})^2) \end{aligned} \quad (4.30)$$

起爆后, 烟幕云团以 3m/s 匀速下沉, 水平坐标保持起爆点的水平位置。第 j 个烟幕云团中心在 t 时刻的位置坐标可表示为:

$$C_j(t) = (17800 + vt_j\cos\theta, vt_j\sin\theta, 1800 - 4.9 \times \Delta t_{burst,j}^2 - 3 \times (t_j - t_{burst,j})) \quad (4.31)$$

4.3.2 优化模型的建立

4.3.2.1 目标函数:

目标函数为最大化总有效遮蔽时间:

对于每个 j 定 $T_{covered,j}$, 表示导弹有效遮蔽的时间段指示:

$$\begin{aligned} f_{covered} &= \begin{cases} 0, \text{非遮蔽时刻} \\ 1, t_i \text{时刻遮蔽} \end{cases} \\ F_{covered} &= \max\{f_j_{covered}\} \\ \max T_{covered,j} &= \int_{t_{start}}^{t_{start}+20} F_{covered}(t)dt \end{aligned} \quad (4.32)$$

取并集:

$$T_{covered} = \cup_{j=1}^3 T_{covered,j} \quad (4.33)$$

对每一个时刻每一架飞机进行遮蔽的 0-1 判断, 只要一架飞机完成真的判断, 即认为此步长时刻遮蔽有效, 对相应飞机投放的烟幕弹遮蔽步长时刻的集合取并集得到最大化总有效遮蔽时间。

4.3.2.2 决策变量:

FY1 的飞行方向角度 θ , 无人机在 xOy 平面上的飞行方向与 x 轴正方向的夹角, 飞行方向的单位向量:

$$\vec{u} = (\cos\theta, \sin\theta, 0)$$

FY1 飞行速度 v : 满足条件:

$$70 \leq v \leq 140$$

第 j 枚烟幕干扰弹投放时间 t_{rel} : 无人机 FY1 受领任务后到投放第 j 枚干扰弹经历的时间, 满足条件:

$$t_{rel,j} \geq 0 \quad (4.34)$$

$$t_{rel,j+1} \geq t_{rel,j}; j = 1, 2$$

第 j 枚烟幕干扰弹起爆时间 t_{burst} ：无人机 FY1 受领任务后到第 j 枚干扰弹起爆经历的时间，满足条件：

$$\begin{aligned} t_{burst,j} &\geq t_{rel,j} \\ t_{burst,j+1} &\geq t_{rel,j} + 1 \end{aligned} \quad (4.35)$$

4.3.2.3 约束条件：

无人机速度大小约束：

$$70 \leq v \leq 140$$

投放时间约束：

$$\begin{aligned} t_{rel,j} &\geq 0 \\ t_{rel,j+1} &\geq t_{rel,j}; j = 1, 2 \end{aligned} \quad (4.36)$$

起爆时间约束：

$$1800 - 4.9 \times (t_{burst,j} - t_{rel,j})^2 \geq 0$$

起爆时序先后约束：

$$\begin{aligned} t_{burst,j} &\geq t_{rel,j} \\ t_{burst,j+1} &\geq t_{rel,j} + 1 \end{aligned} \quad (4.37)$$

飞行方向角约束：

$$\theta \in [0, 2\pi)$$

单枚烟幕干扰弹有效时间约束：

$$0 \leq Fit_j = \max \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt < 20 \quad (4.38)$$

联合遮挡约束：

任一烟幕云团球的球心点 $C_j(t)$ 到空间直线 M_1T 的距离小于球的半径

$$\begin{aligned} 0 \leq d_1 &= \frac{\begin{vmatrix} i & j & k \\ x_1 - x_0 & y_1 - y_0 & z_1 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10 \\ or \ 0 \leq d_2 &= \frac{\begin{vmatrix} i & j & k \\ x_2 - x_0 & y_2 - y_0 & z_2 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10 \\ or \ 0 \leq d_3 &= \frac{\begin{vmatrix} i & j & k \\ x_3 - x_0 & y_3 - y_0 & z_3 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10 \end{aligned} \quad (4.39)$$

总结优化模型：

$$\max \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt \quad (4.40)$$

$$s. t. \left\{ \begin{array}{l} 70 \leq v \leq 140 \\ \theta \in [0, 2\pi) \\ t_{rel,j} \geq 0 \\ t_{rel,j+1} \geq t_{rel,j} + 1 \\ 1800 - 4.9 \times (t_{burst,j} - t_{rel,j})^2 \geq 0 \\ t_{burst,j} \geq t_{rel,j} \\ t_{burst,j+1} \geq t_{rel,j} + 1 \\ 0 \leq Fit_j = \max \left\{ \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt < 20 \right. \\ \left. 0 \leq d = \frac{\begin{vmatrix} i & j & k \\ x_1 - x_0 & y_1 - y_0 & z_1 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10 \right. \end{array} \right.$$

4.3.3 基于差分进化算法改良遗传算法的模型求解

烟幕干扰弹投放策略的优化是一个涉及多变量、非线性约束及时间窗口耦合的复杂问题。以单无人机投放三枚烟幕弹为例，决策变量包括飞行方向、速度、投放时间及起爆延迟，目标函数为真目标有效遮蔽时间的最大化，约束条件涵盖相邻弹投放间隔 ≥ 1 秒、无人机速度在 70~140m/s 范围内等。传统遗传算法（GA）虽能通过群体搜索找到可行解，但在高维度解空间中易因种群多样性衰减陷入局部最优^[4]，可能找到某两枚弹遮蔽时间较长但第三枚与前两枚重叠严重的解，无法实现总遮蔽时间的最大化。这种局限性使其难以满足烟幕干扰对全局最优解的需求。

差分进化算法（DE）作为一种基于种群的启发式优化算法，其核心优势在于利用父代个体之间的差异信息生成变异向量，而非随机扰动。这种定向变异机制能更高效地探索解空间的未知区域，显著增强全局寻优能力，尤其适合求解多维、非线性的全局优化问题^[4]。在烟幕弹投放时序优化中，DE 能通过父代个体的差分向量，引导变异方向向“投放间隔合理、空间分布覆盖导弹轨迹，遮挡时间接力”的区域进化，避免 GA 的随机变异导致的盲目搜索。这种特性使其能有效弥补 GA 易陷入局部最优的缺陷，且能够加快其求解速度，成为烟幕干扰弹投放策略优化的理想补充。

为充分发挥两者的优势，本文提出一种基于差分算法的优化遗传算法（DEGA）^[5]，将 DE 的差分变异机制融入 GA 框架。该算法保留了 GA 的群体搜索策略与基因信息交换特性，同时引入 DE 的差分变异，通过父代个体的差分向量生成变异向量，增强种群的多样性与全局寻优能力。在烟幕干扰弹投放策略优化中，DEGA 的具体应用体现在：种群初始化时采用反向数组法生成初始个体，增加种群多样性；变异步骤用 DE 的差分向量生成新个体，利用三个父代个体的差异信息，生成更优的投放时间与起爆延迟组合，而非 GA 的随机扰动；选择与交叉操作结合 GA 的精英保留，保留适应度高的个体，与 DE 的贪心选择，保留更优的子代个体，确保种群向更优区域进化。

以单无人机投放三枚烟幕弹为例，DEGA 通过差分变异机制快速探索解空间，找到满足 1 秒投放间隔约束且总遮蔽时间最长的投放策略。与单纯 GA 相比，DEGA 能有效跳出“局部最优陷阱”，避免三枚弹因投放间隔过近导致的云团重叠，或间隔过远导致的遮蔽间隙，实现总遮蔽时间的最大化。在多无人机协同干扰问题中，DEGA 能更好地处理空间协同约束，通过全局寻优找到各无人机的最优飞行参数与投放时序，形成连续的

遮蔽区域，提升全局干扰效果。
思路流程如下所示：

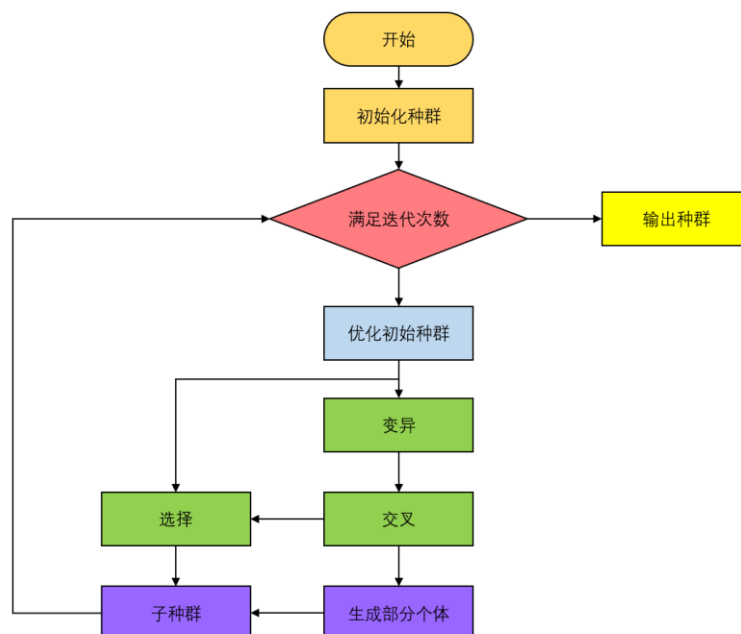


图4.9 DEGA 思路流程图

经过求解，得到最大有效遮蔽时间(并集)为 **6.46s**。相关运行过程可视化如下图所示：

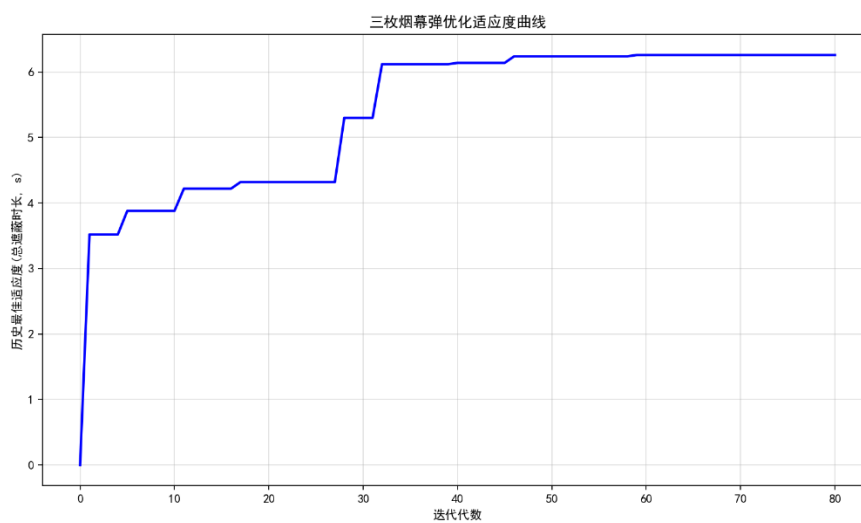


图4.10 烟幕干扰弹优化适应度曲线图

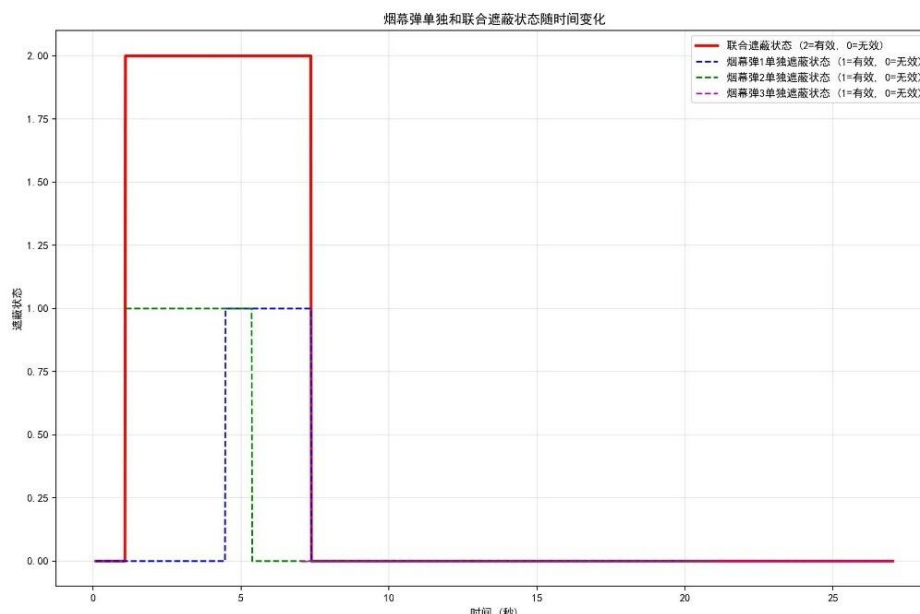


图4.11 烟幕干扰弹单独和联合曲线图

4.3.4 模型的求解

表4.3 问题三结果

参数	数值
相对 x 轴飞机运动方向角	4.999922°
1 号烟幕干扰弹投放点位置	(17800.4, 0.00346, 1800)
2 号烟幕干扰弹投放点位置	(17925.31, 10.96318, 1800)
3 号烟幕干扰弹投放点位置	(18374.25, 50.23937, 1800)
1 号烟幕干扰弹起爆点位置	(17812.57, 1.099433, 1799.951)
2 号烟幕干扰弹起爆点位置	(17937.84, 12.05916, 1799.951)
3 号烟幕干扰弹起爆点位置	(18681.7, 77.13733, 1770.485)
1 号烟幕干扰弹有效遮蔽时间	2.88s
2 号烟幕干扰弹有效遮蔽时间	4.26s
3 号烟幕干扰弹有效遮蔽时间	0s
有效遮蔽总时长	6.46s

4.4 问题四的模型建立与求解

4.4.1 运动模型的建立

4.4.1.1 导弹 M1 的运动模型

导弹 M1 的运动方式为匀速直线运动。在该模型中，核心假设导弹在飞行过程中不受任何外力干扰,如重力、空气阻力或控制指令等，其速度矢量保持恒定，运动轨迹为一条直线。

导弹 M1 初始位置：

$$M_{1,0} = (20000, 0, 2000)$$

飞行速度大小为 300m/s，飞行方向指向假目标原点 O 。从导弹初始位置指向原点的单位向量可表示为 $-e_x$ ，其中负号表示方向指向坐标原点，则导弹飞行速度向量可以表示为

$$\vec{u}_m = \frac{O - M_{1,0}}{\|O - M_{1,0}\|} = -300e_x \quad (4.41)$$

任意时刻导弹 M1 运动方程可表示为:

$$M_1(t) = M_{1,0} + \vec{u}_m \cdot t = (20000 - 300t, 0, 2000) \quad (4.42)$$

4.4.1.2 无人机的运动模型

无人机的运动采用等高度匀速直线运动模型描述,其核心假设为飞行过程中保持高度、以恒定速度沿直线飞行,忽略空气阻力、风力干扰及机动变化等因素。根据题目约束,无人机受领任务后可瞬时调整飞行方向,随后航向与速度一旦确定即保持不变,仅通过飞行方向(水平角度)与速度的优化,控制烟幕弹投放点。

无人机初始位置:

$$\begin{aligned} P_{1,0} &= (17800, 0, 1800) \\ P_{2,0} &= (12000, 1400, 1400) \\ P_{3,0} &= (6000, -3000, 700) \end{aligned}$$

无人机飞行的水平高度不变:

$$u_{i,z} = 0$$

无人机飞行的单位方向向量:

$$\|\vec{u}_i\| = \sqrt{u_{i,x}^2 + u_{i,y}^2 + u_{i,z}^2} = 1, \vec{u}_i = (\cos\theta, \sin\theta, 0) \quad (4.43)$$

无人机飞行速度约束:

$$70 \leq v \leq 140$$

无人机投放时刻位置:

$$\begin{aligned} P_i(t) &= P_{i,0} + vt_{rel}\vec{u} \\ P_1(t) &= (17800 + vt_{rel}\cos\theta, vt_{rel}\sin\theta, 1800) \\ P_2(t) &= (12000 + vt_{rel}\cos\theta, vt_{rel}\sin\theta + 1400, 1400) \\ P_3(t) &= (6000 + vt_{rel}\cos\theta, vt_{rel}\sin\theta - 3000, 700) \end{aligned} \quad (4.44)$$

4.4.1.3 烟幕弹投放与起爆位置模型

烟幕弹的投放与起爆位置模型基于无人机的等高度匀速直线运动构建:无人机受领任务后,沿调整后的方向以恒定速度飞行, t_{rel} 时刻投放一枚烟幕弹。所有烟幕弹投放后, xOy 平面方向与飞机速度方向一致,竖直方向做自由落体运动;爆开后,在竖直方向匀速下落, xOy 平面方向速度归零。

第*i*架无人机烟幕弹投放时间:

$$t_{rel,j}$$

第*i*架无人机烟幕弹起爆时间:

$$t_{burst,j} = t_{rel,j} + \Delta t_{burst,j} \quad (4.45)$$

此时起爆点位置方程可描述为:

$$\begin{aligned} P_{burst,j} &= P_{1,0} + vt_{burst,j}\vec{u} - 0.5g\Delta t_{burst,j}^2 \cdot e_z \\ &= (17800 + vt_{burst,j}\cos\theta, vt_{burst,j}\sin\theta, 1800 - 4.9 \times (t_{burst,j} - t_{rel,j})^2) \end{aligned} \quad (4.46)$$

起爆后,烟幕云团以 3m/s 匀速下沉,水平坐标保持起爆点的水平位置。第*j*个烟幕云团中心在*t*时刻的位置坐标可表示为:

$$\begin{aligned} C_j(t) &= (17800 + vt_j\cos\theta, vt_j\sin\theta, 1800 - 4.9 \times \Delta t_{burst,j}^2 - 3 \times (t \\ &\quad - t_{burst,j})) \end{aligned} \quad (4.47)$$

4.4.2 三机投弹的遮挡时间优化模型的建立:

4.4.2.1 目标函数

目标函数为最大化总有效遮蔽时间：

对于每个 j 定 $T_{covered,j}$ ，表示导弹有效遮蔽的时间段指示：

$$\begin{aligned} f_{covered} &= \begin{cases} 0, & \text{非遮蔽时刻} \\ 1, & t_i \text{时刻遮蔽} \end{cases} \\ F_{covered} &= \max\{f_j, covered\} \\ \max T_{covered,j} &= \int_{t_{start}}^{t_{start}+20} F_{covered}(t)dt \end{aligned} \quad (4.48)$$

取并集：

$$T_{covered} = \cup_{j=1}^3 T_{covered,j}$$

对每一个时刻每一架飞机进行遮蔽的 0-1 判断，只要一架飞机完成真的判断，即认为此步长时刻遮蔽有效，对相应飞机投放的烟幕弹遮蔽步长时刻的集合取并集得到最大化总有效遮蔽时间。

4.4.2.2 决策变量

无人机的飞行方向角度 θ ，无人机在 xOy 平面上的飞行方向与 x 轴正方向的夹角，飞行方向的单位向量：

$$\vec{u} = (\cos\theta, \sin\theta, 0)$$

无人机飞行速度 v ：满足条件：

$$70 \leq v \leq 140$$

第 i 架无人机投放的一枚烟幕干扰弹即为第 j 枚烟幕干扰弹，此时 $i = j$ 。

第 j 枚烟幕干扰弹投放时间 t_{rel} ：无人机 FY 受领任务后到投放第 j 枚干扰弹经历的时间，满足条件：

$$t_{rel,j} \geq 0 \quad (4.49)$$

第 j 枚烟幕干扰弹起爆时间 t_{burst} ：无人机 FY1 受领任务后到第 j 枚干扰弹起爆经历的时间，满足条件：

$$t_{burst,j} \geq t_{rel,j} \quad (4.50)$$

4.4.2.3 约束条件

无人机速度大小约束：

$$70 \leq v \leq 140$$

投放时间约束：

$$t_{rel,j} \geq 0$$

起爆时间约束：

$$1800 - 4.9 \times (t_{burst,j} - t_{rel,j})^2 \geq 0$$

起爆时序先后约束：

$$t_{burst,j} \geq t_{rel,j}$$

飞行方向角约束：

$$\theta \in [0, 2\pi)$$

单枚烟幕干扰弹有效时间约束：

$$0 \leq Fit_j = \max \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt < 20$$

联合遮挡约束：

任一烟幕云团球的球心点 $C_j(t)$ 到空间直线 M_1T 的距离小于球的半径即可

$$\begin{aligned}
0 \leq d_1 &= \frac{\begin{vmatrix} i & j & k \\ x_1 - x_0 & y_1 - y_0 & z_1 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10 \\
\text{or } 0 \leq d_2 &= \frac{\begin{vmatrix} i & j & k \\ x_2 - x_0 & y_2 - y_0 & z_2 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10 \\
\text{or } 0 \leq d_3 &= \frac{\begin{vmatrix} i & j & k \\ x_3 - x_0 & y_3 - y_0 & z_3 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10
\end{aligned} \tag{4.51}$$

总结优化模型：

$$\begin{aligned}
& \max \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt \\
& \text{s. t. } \left\{ \begin{aligned}
& 70 \leq v \leq 140 \\
& \theta \in [0, 2\pi) \\
& t_{rel,j} \geq 0 \\
& 1800 - 4.9 \times (t_{burst,j} - t_{rel,j})^2 \geq 0 \\
& t_{burst,j} \geq t_{rel,j} \\
& 0 \leq Fit_j = \max \left\{ \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt < 20 \right. \\
& 0 \leq d_1 = \frac{\begin{vmatrix} i & j & k \\ x_1 - x_0 & y_1 - y_0 & z_1 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10 \\
& \text{or } 0 \leq d_2 = \frac{\begin{vmatrix} i & j & k \\ x_2 - x_0 & y_2 - y_0 & z_2 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10 \\
& \text{or } 0 \leq d_3 = \frac{\begin{vmatrix} i & j & k \\ x_3 - x_0 & y_3 - y_0 & z_3 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10
\end{aligned} \right. \tag{4.52}
\end{aligned}$$

4.4.3 分层优化下模型的求解：

第四问在第二问的基础上，多增加了两家无人机 FLY2 与 FLY3 坐标，想要最大化目标函数为三架无人机所投烟雾弹产生遮蔽时间段的并集，则期望三架无人机接力产生遮蔽，考虑到与第二问的关联性，该问采用分层优化的思想进行求解，借用第二问的求解方式分别求解出 FLY2 FLY3 在单机运行时对导弹 M1 的遮挡效果，得到了这两架无人机在第二问的环境下所能达到的最优遮挡效果以及遮挡时间段，记录此时 FLY2 FLY3 的相关参数，并以此时得出的结果简单求并集得出参考基准。

接下来进行精细化的求解，固定 FLY1 的飞行参数，即只是对 FLY2 FLY3 进行相关参数的优化，根据上一步所得出的 FLY2 FLY3 的飞行速度，飞行方向为初始化数据定调，即初始化数据只在这附近做微小扰动，沿用第三问的 DEGA 算法进行求解，从运行结果来看，该分层优化策略大大减少了代码运行时间，提高了求解效率。

相关运行过程可视化如下图所示：

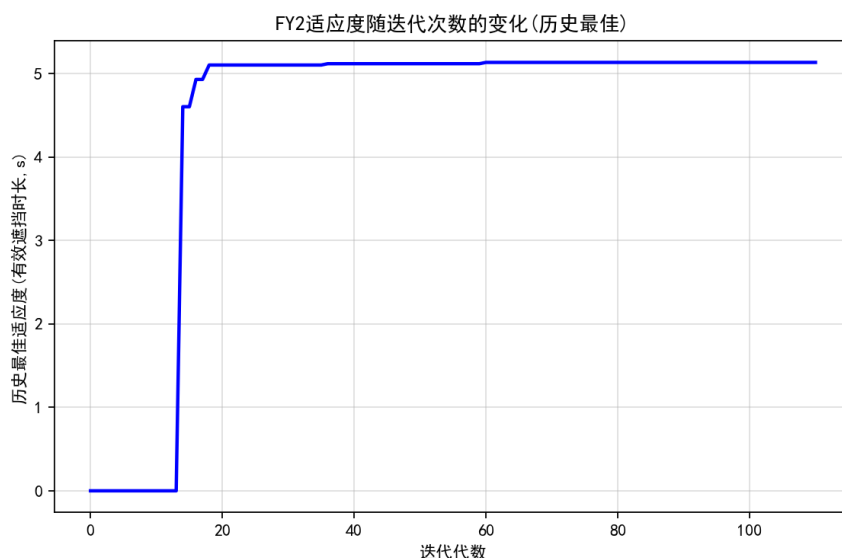


图4.12 FY2 适应度曲线图

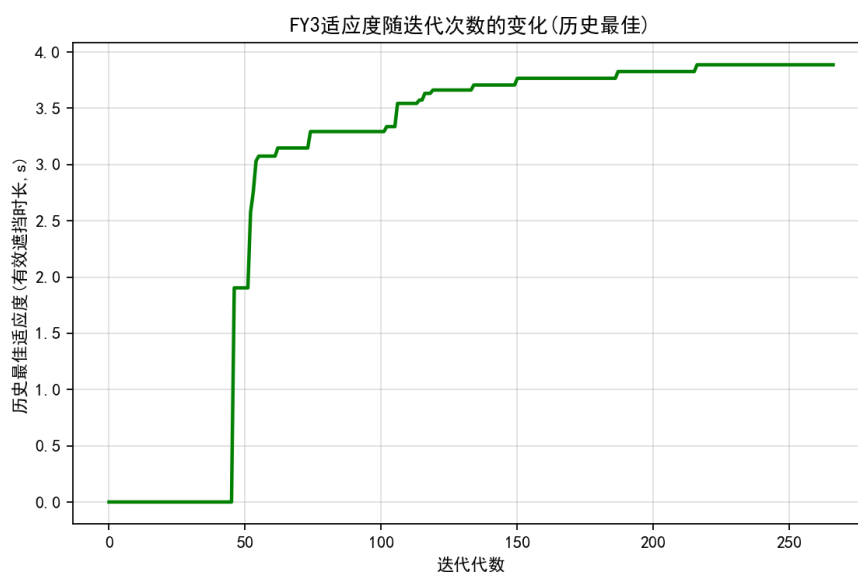


图4.13 FY3 适应度曲线图

表4.4 问题四结果

参数	数值
1 号无人机相对 x 轴运动方向角	5.4408°
1 号无人机运动速度	111.6373m/s
1 号无人机投放延迟时间	0.5371s
1 号无人机起爆延迟时间	0.4851s
1 号无人机烟幕干扰弹投放点位置	(17859.686, 5.685, 1800)
1 号无人机烟幕干扰弹起爆点位置	(17913.593, 10.819, 1798.847)

1 号无人机烟幕干扰弹有效遮蔽开始时刻	1.232s
1 号无人机烟幕干扰弹有效遮蔽结束时刻	5.812s
1 号无人机烟幕干扰弹有效遮蔽总时长	4.580s
2 号无人机相对 x 轴运动方向角	277.5286°
2 号无人机运动速度	111.7809m/s
2 号无人机投放延迟时间	5.7401s
2 号无人机起爆延迟时间	5.9575s
2 号无人机烟幕干扰弹投放点位置	(12087.828, 735.440, 1400)
2 号无人机烟幕干扰弹起爆点位置	(12178.982, 45.717, 1225.091)
2 号无人机烟幕干扰弹有效遮蔽开始时刻	12.148s
2 号无人机烟幕干扰弹有效遮蔽结束时刻	16.058s
2 号无人机烟幕干扰弹有效遮蔽总时长	3.910s
3 号无人机相对 x 轴运动方向角	74.2358°
3 号无人机运动速度	130.0308m/s
3 号无人机投放延迟时间	23.9248s
3 号无人机起爆延迟时间	0.7118s
3 号无人机烟幕干扰弹投放点位置	(6845.180, -6.053, 700)
3 号无人机烟幕干扰弹起爆点位置	(6870.327, 83.026, 697.517)
3 号无人机烟幕干扰弹有效遮蔽开始时刻	26.017s
3 号无人机烟幕干扰弹有效遮蔽结束时刻	29.117s
3 号无人机烟幕干扰弹有效遮蔽总时长	3.100s
总有效遮蔽时长	11.59s

4.5 问题五的模型建立与求解

4.5.1 运动模型的建立

4.5.1.1 导弹的运动模型

导弹的运动方式为匀速直线运动。在该模型中，核心假设导弹在飞行过程中不受任何外力干扰,如重力、空气阻力或控制指令等，其速度矢量保持恒定，运动轨迹为一条直线。

导弹 M_k 初始位置：

$$\begin{aligned} M_{1,0} &= (20000, 0, 2000) \\ M_{2,0} &= (19000, 600, 2100) \\ M_{3,0} &= (18000, -600, 1900) \end{aligned}$$

飞行速度大小为 300m/s，飞行方向指向假目标原点 O 。从导弹初始位置指向原点的单位向量可表示为 $-e_x$ ，其中负号表示方向指向坐标原点，则导弹飞行速度向量可以表示为

$$\vec{u}_m = \frac{O - M_{k,0}}{\|O - M_{k,0}\|} \quad (4.53)$$

任意时刻导弹 M_k 运动方程可表示为：

$$M_k(t) = M_{k,0} + \vec{u}_m \cdot t \quad (4.54)$$

4.5.1.2 无人机的运动模型

无人机的运动采用等高度匀速直线运动模型描述，其核心假设为飞行过程中保持固定高度、以恒定速度沿直线飞行，忽略空气阻力、风力干扰及机动变化等因素。每架无人机至多投放 3 枚烟幕干扰弹,受领任务后可瞬时调整飞行方向，随后航向与速度一旦

确定即保持不变，仅通过飞行方向（水平角度）与速度的优化，控制烟幕弹投放点。

无人机初始位置：

$$\begin{aligned} P_{1,0} &= (17800, 0, 1800) \\ P_{2,0} &= (12000, 1400, 1400) \\ P_{3,0} &= (6000, -3000, 700) \\ P_{4,0} &= (11000, 2000, 1800) \\ P_{5,0} &= (13000, -2000, 1300) \end{aligned}$$

无人机飞行的水平高度不变：

$$u_{i,z} = 0$$

无人机飞行的单位方向向量：

$$\|\vec{u}_i\| = \sqrt{u_{i,x}^2 + u_{i,y}^2 + u_{i,z}^2} = 1, \vec{u}_i = (\cos\theta, \sin\theta, 0) \quad (4.55)$$

无人机飞行速度约束：

$$70 \leq v \leq 140$$

第*i*架无人机投放第*j*枚烟幕弹时刻位置：

$$\begin{aligned} P_i(t) &= P_{i,0} + vt_{rel,ij}\vec{u} \\ P_1(t) &= (17800 + vt_{rel,1j}\cos\theta, vt_{rel,1j}\sin\theta, 1800) \\ P_2(t) &= (12000 + vt_{rel,2j}\cos\theta, vt_{rel,2j}\sin\theta + 1400, 1400) \\ P_3(t) &= (6000 + vt_{rel,3j}\cos\theta, vt_{rel,3j}\sin\theta - 3000, 700) \\ P_4(t) &= (11000 + vt_{rel,4j}\cos\theta, vt_{rel,4j}\sin\theta + 2000, 1800) \\ P_5(t) &= (6000 + vt_{rel,5j}\cos\theta, vt_{rel,5j}\sin\theta - 2000, 1300) \end{aligned} \quad (4.56)$$

4.5.1.3 烟幕弹投放与起爆位置模型

烟幕弹的投放与起爆位置模型基于无人机的等高度匀速直线运动构建：第*i*架无人机受领任务后，沿调整后的方向以恒定速度飞行， $t_{rel,ij}$ 时刻投放第*j*枚烟幕弹。所有烟幕弹投放后， xOy 平面方向与飞机速度方向一致，竖直方向做自由落体运动；爆开后，在竖直方向匀速下落， xOy 平面方向速度归零。

第*i*架无人机投放第*j*枚烟幕弹时间：

$$t_{rel,ij}$$

第*i*架无人机投放第*j*枚烟幕弹起爆时间：

$$t_{burst,ij} = t_{rel,ij} + \Delta t_{burst,ij} \quad (4.57)$$

此时起爆点位置方程可描述为：

$$\begin{aligned} P_{burst,ij} &= P_{i,0} + vt_{burst,ij}\vec{u} - 0.5g\Delta t_{burst,ij}^2 \cdot e_z \\ &= (17800 + vt_{burst,ij}\cos\theta, vt_{burst,ij}\sin\theta, 1800 - 4.9 \times (t_{burst,ij} - t_{rel,ij})^2) \end{aligned} \quad (4.58)$$

起爆后，烟幕云团以 3m/s 匀速下沉，水平坐标保持起爆点的水平位置。第*i*架无人机投放第*j*个烟幕云团中心在*t*时刻的位置坐标可表示为：

$$\begin{aligned} C_{ij}(t) &= (17800 + vt_{ij}\cos\theta, vt_{ij}\sin\theta, 1800 - 4.9 \times \Delta t_{burst,ij}^2 - 3 \times (t \\ &\quad - t_{burst,ij})) \end{aligned} \quad (4.59)$$

4.5.2 分组策略与分层优化模型的建立与求解

4.5.2.1 目标函数：

首先明确遮挡时间计算原则，考虑三枚导弹独立计算遮蔽时间，汇总成总遮蔽时间，

以此为目标函数：

$$T_{covered} = \sum_{i=1}^3 \Delta T_{covered,i} \quad (4.60)$$

$$\Delta T_{covered,i} = \max \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt$$

4.5.2.2 决策变量：

无人机的飞行方向角度 θ ，无人机在 xOy 平面上的飞行方向与 x 轴正方向的夹角，飞行方向的单位向量：

$$\vec{u} = (\cos\theta, \sin\theta, 0)$$

无人机飞行速度 v ：满足条件：

$$70 \leq v \leq 140:$$

第 j 枚烟幕干扰弹投放时间 t_{rel} ：第 i 架无人机受领任务后到投放第 j 枚干扰弹经历的时间，满足条件：

$$\begin{aligned} t_{rel,ij} &\geq 0 \\ t_{rel,ij+1} &\geq t_{rel,ij}; j = 1, 2 \end{aligned} \quad (4.61)$$

第 j 枚烟幕干扰弹起爆时间 t_{burst} ：第 i 架无人机受领任务后到第 j 枚干扰弹起爆经历的时间，满足条件：

$$\begin{aligned} t_{burst,ij} &\geq t_{rel,ij} \\ t_{burst,ij+1} &\geq t_{rel,ij} + 1 \end{aligned} \quad (4.62)$$

4.5.2.3 约束条件：

无人机速度大小约束：

$$70 \leq v \leq 140$$

投放时间约束：

$$\begin{aligned} t_{rel,ij} &\geq 0 \\ t_{rel,ij+1} &\geq t_{rel,ij}; i = 1, 2, 3, 4, 5; j = 1, 2 \end{aligned} \quad (4.63)$$

起爆时间约束：

$$1800 - 4.9 \times (t_{burst,ij} - t_{rel,ij})^2 \geq 0$$

起爆时序先后约束：

$$\begin{aligned} t_{burst,ij} &\geq t_{rel,ij} \\ t_{burst,ij+1} &\geq t_{rel,ij} + 1 \end{aligned} \quad (4.64)$$

飞行方向角约束：

$$\theta \in [0, 2\pi)$$

单枚烟幕干扰弹有效时间约束：

$$0 \leq Fit_j = \max \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt < 20 \quad (4.65)$$

烟幕云团球的球心点 $C_{ij}(t)$ 到空间直线 M_iT 的距离小于球的半径

$$0 \leq d = \frac{\begin{vmatrix} i & j & k \\ x_1 - x_0 & y_1 - y_0 & z_1 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10 \quad (4.66)$$

总结优化模型：

$$\begin{aligned}
 & \max \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt \\
 & s. t. \left\{ \begin{aligned}
 & 70 \leq v \leq 140 \\
 & \theta \in [0, 2\pi) \\
 & t_{rel,j} \geq 0 \\
 & t_{rel,j+1} \geq t_{rel,j} + 1 \\
 & 1800 - 4.9 \times (t_{burst,j} - t_{rel,j})^2 \geq 0 \\
 & t_{burst,j} \geq t_{rel,j} \\
 & t_{burst,j+1} \geq t_{rel,j} + 1 \\
 & 0 \leq Fit_j = \max \left\{ \sum_{i=0}^{\frac{20}{\Delta t}} \int_{t_i}^{t_{i+1}} f(u, v, t, \tau, T) dt < 20 \right. \\
 & 0 \leq d = \frac{\begin{vmatrix} i & j & k \\ x_1 - x_0 & y_1 - y_0 & z_1 - z_0 \\ l & m & n \end{vmatrix}}{\sqrt{l^2 + m^2 + n^2}} < 10
 \end{aligned} \right. \quad (4.67)
 \end{aligned}$$

由于此问决策变量过多，求解时间空间复杂度均达到一较高的值，则采用分组策略，为每一无人机分配具体遮蔽任务，以无人机与导弹初始位置为基础，构建适应度矩阵，以此明确无人机对应的任务，要求每个无人机分配到两颗导弹的主次任务，同时确保每颗导弹都有对应的无人机进行匹配，匹配分组完成后，以分组条件为一硬性约束开始优化求解。

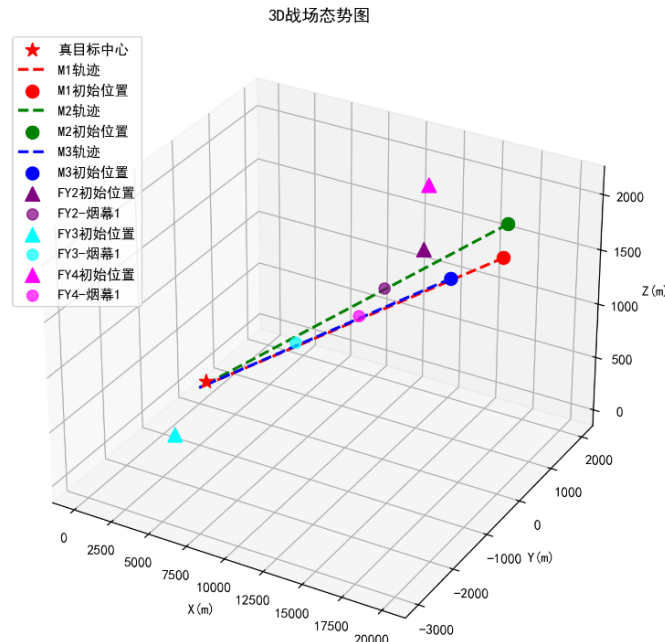


图4.14 3D 战场图

采用分层优化的方式，分配各无人机子任务，各子任务进行单独优化，采用差分进化算法进行寻优，在子任务优化完成后，进行任务汇总，得到最终的投放策略如下表所示（部分结果）：

表4.5 问题五结果

参数	数值
1 号无人机投放第 1 枚干扰 M1 烟幕弹有效遮蔽时长	2.48s
1 号无人机投放第 2 枚干扰 M1 烟幕弹有效遮蔽时长	1.27s
1 号无人机投放第 3 枚干扰 M1 烟幕弹有效遮蔽时长	0.66s
2 号无人机投放第 1 枚干扰 M1 烟幕弹有效遮蔽时长	3.88s
2 号无人机投放第 2 枚干扰 M1 烟幕弹有效遮蔽时长	0s
2 号无人机投放第 3 枚干扰 M1 烟幕弹有效遮蔽时长	0s
3 号无人机投放第 1 枚干扰 M3 烟幕弹有效遮蔽时长	0.95s
3 号无人机投放第 2 枚干扰 M3 烟幕弹有效遮蔽时长	0.63s
3 号无人机投放第 3 枚干扰 M3 烟幕弹有效遮蔽时长	1.26s
4 号无人机投放第 1 枚干扰 M2 烟幕弹有效遮蔽时长	0s
4 号无人机投放第 2 枚干扰 M2 烟幕弹有效遮蔽时长	3.81s
4 号无人机投放第 3 枚干扰 M2 烟幕弹有效遮蔽时长	2.63s
5 号无人机投放第 1 枚干扰 M1 烟幕弹有效遮蔽时长	1.24s
5 号无人机投放第 2 枚干扰 M1 烟幕弹有效遮蔽时长	0.61s
5 号无人机投放第 3 枚干扰 M1 烟幕弹有效遮蔽时长	1.85s
总有效遮蔽时长	21.29s

五、模型的检验与分析

5.1 基于动态调参的灵敏度检验：

本文中使用频率最高的是差分进化算法,本节将对差分进化算法进行深度分析与检验，重点包括动态调参下的灵敏度检验。

依次对如下参数进行调整，种群大小（POP_SIZE）、最大迭代次数（MAX_GEN）、缩放因子（F0）和交叉概率（CR）。定义参数范围：对于每个参数，选择一个范围，例如 POP_SIZE 从 20 到 100, MAX_GEN 从 50 到 200, F0 从 0.5 到 1.0, CR 从 0.5 到 1.0。

绘制每个参数变化时最优适应度值的变化曲线，集成到一张图中。

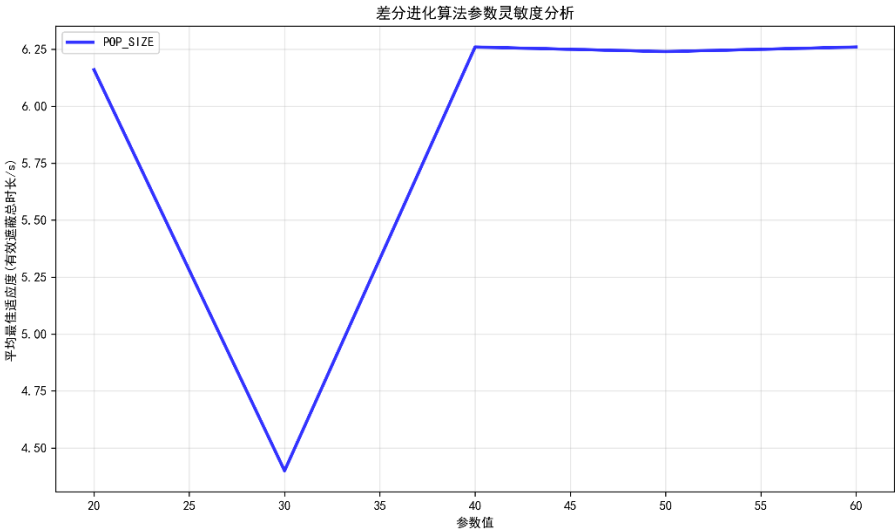


图5.1 差分进化算法灵敏度曲线

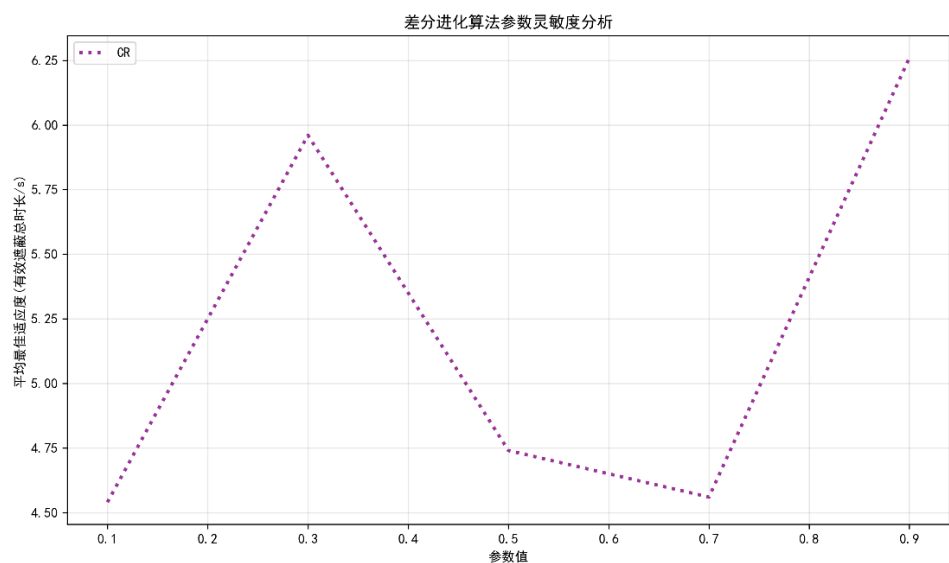


图5.2 差分进化算法灵敏度曲线

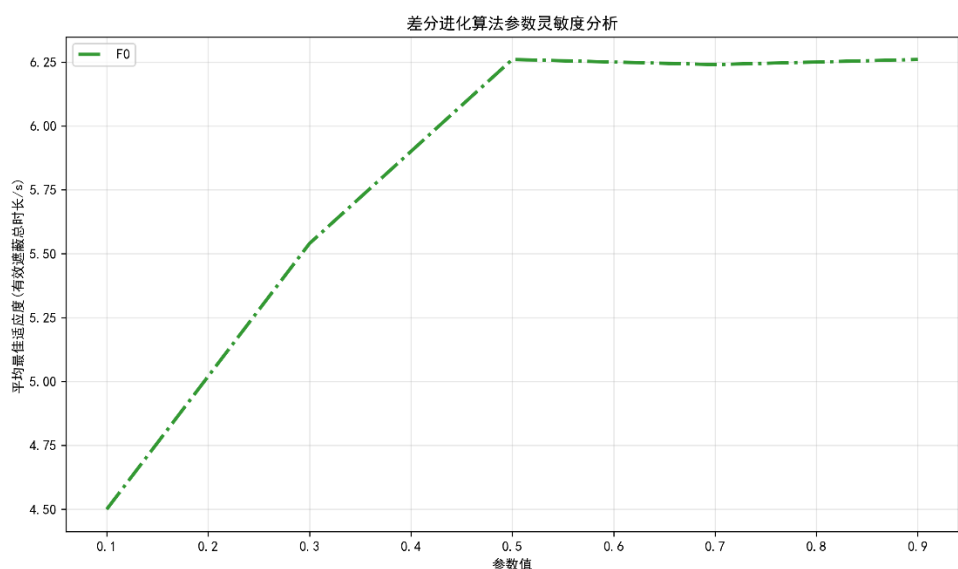


图5.3 差分进化算法灵敏度曲线

六、模型评价与推广

6.1 模型的优点

(1) 在第三问的模型中引入差分进化算法，“差分变异”能够生成新的情况，扩大了搜索范围，正好对应了该问中投放多枚烟幕弹的情景，结合遗传算法的选择，交叉，变异操作，有效防止了陷入局部最优解。

(2) 在第四问模型中引入了分层优化的思想，分设多层，限定参数范围，有效增加了代码运行效率。

(3) 模型充分考虑了本题设问的连贯性以相关性，在求解问题时模型的建立，算法的选用不只是针对于孤立的子问题，而是联系各个问题综合选用最适合的算法。

6.2 模型的缺点

(1) 为了保证效率，在进行寻优过程中限定了迭代次数，在一定程度上可能降低了精确度。

6.3 模型的推广

本文建立的基于差分进化改良遗传算法的干扰弹投放优化模型具备良好可拓展性。其中遮挡判定模型的可行性，从单机单导弹自然扩展到多机多导弹的优化策略以及算法连续性，可以在一定程度上解决实际情景下的干扰弹投放问题。目标函数可由“遮蔽并集最大”扩展到“空窗最小、能耗更低”等多目标形式，约束也可叠加禁飞区与安全间隔。整体框架对更复杂的协同场景保持通用性和可落地性^[6]。

七、参考文献

- [1] 王玄玉,潘功配.几种烟幕对 CO₂ 激光的衰减性能研究[J].激光与红外,2006(02):144-146.
- [2] DeepSeek,DeepSeek-R1,深度求索 (DeepSeek) , 2025-09-05.
- [3] 汪士琦,李加文,何杨青,等.基于改进遗传算法适应度评价函数的路径规划[J].机械设计,2025,42(09):86-93.
- [4] 陈强.差分进化算法改进及定位应用[J].北斗与空间信息应用技术,2025(03):28-31.
- [5] 吴斌.基于智能优化算法的 WSN 节点定位技术研究[D].杭州电子科技大学,2019.
- [6] ChatGPT; GPT-5 Thinking; OpenAI; 2025-09-07.

八、附录

附录

附录一：

第一小问代码 1 (用于测算并求出合适的采样规模)

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from scipy.optimize import minimize
import time

plt.rcParams["font.family"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

np.random.seed(42)

R_CYL = 7
H_CYL = 10
MISSILE_START = np.array([20000, 0, 2000])
CYL_CENTER = np.array([0, 200, 5])
CYL_BOTTOM = np.array([0, 200, 0])
CYL_TOP = np.array([0, 200, 10])

def dist_pt_to_line(pt, line_pt1, line_pt2):
    vec1 = pt - line_pt1
    vec2 = line_pt2 - line_pt1
    cross_val = np.linalg.norm(np.cross(vec1, vec2))
    line_len = np.linalg.norm(vec2)
    return cross_val / line_len

def gen_smoke_center():
    while True:
        rad = np.random.uniform(10, 50)
        theta_val = np.random.uniform(0, 2*np.pi)
        phi_val = np.random.uniform(0, np.pi)

        x_val = rad * np.sin(phi_val) * np.cos(theta_val)
        y_val = rad * np.sin(phi_val) * np.sin(theta_val)
        z_val = rad * np.cos(phi_val)

        smoke_ctr = CYL_CENTER + np.array([x_val, y_val, z_val])

        dist_bottom = dist_pt_to_line(smoke_ctr, MISSILE_START, CYL_BOTTOM)
        dist_top = dist_pt_to_line(smoke_ctr, MISSILE_START, CYL_TOP)
        dist_missile = np.linalg.norm(smoke_ctr - MISSILE_START)
        dist_to_center = np.linalg.norm(MISSILE_START - CYL_CENTER)

        if (dist_bottom < 10 and
            dist_top > 10 and
            dist_missile < dist_to_center):
            return smoke_ctr

def gen_sample_pts(a_param, b_param):
    samp_pts = []

    for layer_idx in range(b_param + 1):
        z_val = layer_idx * H_CYL / b_param

        for i_idx in range(a_param):
```



```

        angle_val = 2 * np.pi * i_idx / a_param
        x_val = R_CYL * np.cos(angle_val)
        y_val = 200 + R_CYL * np.sin(angle_val)

        samp_pts.append([x_val, y_val, z_val])

    return np.array(samp_pts)

def calc_coverage_ratio(smoke_ctr, samp_pts):
    cover_cnt = 0
    total_cnt = len(samp_pts)

    for pt in samp_pts:
        dist_val = dist_pt_to_line(smoke_ctr, MISSILE_START, pt)

        if dist_val <= 10:
            cover_cnt += 1

    return cover_cnt / total_cnt

def main():
    smoke_ctr = gen_smoke_center()
    print(f"烟雾球中心坐标: {smoke_ctr}")

    delta_vals = np.arange(0.1, 3.1, 0.1)
    cover_ratios = []
    a_vals = []
    b_vals = []

    for delta in delta_vals:
        a_param = int(2 * np.pi * R_CYL * delta)
        b_param = int(H_CYL * delta)

        if a_param == 0:
            a_param = 1
        if b_param == 0:
            b_param = 1

        a_vals.append(a_param)
        b_vals.append(b_param)

        samp_pts = gen_sample_pts(a_param, b_param)

        ratio_val = calc_coverage_ratio(smoke_ctr, samp_pts)
        cover_ratios.append(ratio_val)

    print(f"delta={delta:.1f}, a={a_param}, b={b_param}, 采样遮挡率={ratio_val:.4f}")

    plt.figure(figsize=(12, 6))

    plt.subplot(1, 2, 1)
    plt.plot(delta_vals, cover_ratios, 'bo-')
    plt.xlabel('Delta')
    plt.ylabel('采样遮挡率')
    plt.title('采样遮挡率随 Delta 变化')
    plt.grid(True)

    total_samps = [a * (b + 1) for a, b in zip(a_vals, b_vals)]
    plt.subplot(1, 2, 2)
    plt.plot(total_samps, cover_ratios, 'ro-')
    plt.xlabel('总样本点数')
    plt.ylabel('采样遮挡率')
    plt.title('采样遮挡率随样本点数变化')
    plt.grid(True)

```

```

plt.tight_layout()
plt.savefig('coverage_ratio_vs_delta.png', dpi=300)
plt.show()

converge_delta = None
for i in range(1, len(delta_vals)-1):
    if (abs(cover_ratios[i] - cover_ratios[-1]) < 0.01 and
        abs(cover_ratios[i-1] - cover_ratios[-1]) < 0.01):
        converge_delta = delta_vals[i]
        converge_a = a_vals[i]
        converge_b = b_vals[i]
        converge_ratio = cover_ratios[i]
        break

if converge_delta is not None:
    print(f"\n收敛点: delta={converge_delta:.1f}, a={converge_a},
b={converge_b}")
    print(f"对应的采样遮挡率: {converge_ratio:.4f}")
    print(f"总样本点数: {converge_a * (converge_b + 1)}")
else:
    print("未找到明显的收敛点,可能需要增加 delta 范围")

if converge_delta is not None:
    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection='3d')

    ax.scatter(MISSILE_START[0], MISSILE_START[1], MISSILE_START[2],
c='r', marker='^', s=100, label='导弹')

    ax.scatter(smoke_ctr[0], smoke_ctr[1], smoke_ctr[2], c='g',
marker='o', s=100, label='烟雾球中心')

    samp_pts = gen_sample_pts(converge_a, converge_b)
    ax.scatter(samp_pts[:, 0], samp_pts[:, 1], samp_pts[:, 2], c='b',
marker='.', s=10, alpha=0.5, label='目标样本点')

    ax.scatter(CYL_CENTER[0], CYL_CENTER[1], CYL_CENTER[2], c='k',
marker='x', s=100, label='圆柱中心')

    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    ax.set_zlabel('Z')
    ax.legend()
    plt.title('烟雾球、导弹和目标样本点的空间分布')
    plt.savefig('spatial_distribution.png', dpi=300)
    plt.show()

if __name__ == "__main__":
    main()

```

附录二:

第一小问代码 2 (用于求解烟幕的有效遮挡时长)

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

plt.rcParams["font.family"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

```

```

GRAVITY = 9.8
MISSILE_VEL = 300
SINK_VEL = 3
EFF_RADIUS = 10
EFF_TIME = 20
DT = 0.001

CYL_CENTER = np.array([0, 200, 5])
CYL_RADIUS = 7
CYL_HEIGHT = 10

A_DEF = 52
B_DEF = 12

M1_START = np.array([20000, 0, 2000])
DECOY_TARGET = np.array([0, 0, 0])
FY1_START = np.array([17800, 0, 1800])
UAV_VEL = 120

uav_dir = DECOY_TARGET[:2] - FY1_START[:2]
uav_dir_norm = np.linalg.norm(uav_dir)
uav_dir_unit = uav_dir / uav_dir_norm
uav_vel_vec = UAV_VEL * np.array([uav_dir_unit[0], uav_dir_unit[1], 0])

TASK_START = 0
DROP_DELAY = 1.5
BURST_DELAY = 3.6
DROP_TIME = TASK_START + DROP_DELAY
BURST_TIME = DROP_TIME + BURST_DELAY
END_TIME = BURST_TIME + EFF_TIME

missile_dir = DECOY_TARGET - M1_START
missile_dir_norm = np.linalg.norm(missile_dir)
missile_dir_unit = missile_dir / missile_dir_norm
missile_vel_vec = MISSILE_VEL * missile_dir_unit

def dist_pt_to_segment(pt, seg_pt1, seg_pt2):
    vec1 = pt - seg_pt1
    vec2 = seg_pt2 - seg_pt1
    cross_val = np.linalg.norm(np.cross(vec1, vec2))
    dot_val = np.dot(vec1, vec2)
    seg_len = np.linalg.norm(vec2)
    if dot_val > 0 and seg_len > 1e-10:
        return cross_val / seg_len
    else:
        return np.linalg.norm(pt - seg_pt1)

def gen_sample_pts(a_param, b_param):
    samp_pts = []
    for layer in range(b_param + 1):
        z_val = layer * CYL_HEIGHT / b_param

        for i in range(a_param):
            angle_val = 2 * np.pi * i / a_param
            x_val = CYL_RADIUS * np.cos(angle_val)
            y_val = 200 + CYL_RADIUS * np.sin(angle_val)
            samp_pts.append([x_val, y_val, z_val])

    return np.array(samp_pts)

def is_eff_cover(missile_pos, cloud_pos, samp_pts, a_param, b_param):
    dist_to_center = dist_pt_to_segment(cloud_pos, missile_pos, CYL_CENTER)
    if dist_to_center > EFF_RADIUS:
        return False

```

```

        for pt in samp_pts:
            dist_to_sample = dist_pt_to_segment(cloud_pos, missile_pos, pt)
            if dist_to_sample > EFF_RADIUS:
                return False
        return True

def main():
    a_param = A_DEF
    b_param = B_DEF
    samp_pts = gen_sample_pts(a_param, b_param)
    print(f"使用离散化参数: a={a_param}, b={b_param}, 总样本点数: {len(samp_pts)}")

    uav_pos_drop = FY1_START + uav_vel_vec * DROP_TIME

    smoke_start_vel = uav_vel_vec

    time_to_burst = BURST_TIME - DROP_TIME

    horiz_disp = smoke_start_vel * time_to_burst
    vert_disp = 0.5 * np.array([0, 0, -GRAVITY]) * time_to_burst**2

    burst_pos = uav_pos_drop + horiz_disp + vert_disp
    print(f"起爆点位置: {burst_pos}")

    time_pts = np.arange(BURST_TIME, END_TIME + DT, DT)
    cover_states = []
    cover_start = None
    cover_periods = []

    for t in time_pts:
        missile_pos = M1_START + missile_vel_vec * t

        time_after_burst = t - BURST_TIME
        cloud_pos = np.array([burst_pos[0], burst_pos[1], burst_pos[2] -
SINK_VEL * time_after_burst])

        effective = is_eff_cover(missile_pos, cloud_pos, samp_pts, a_param,
b_param)
        cover_states.append(effective)

        if effective:
            if cover_start is None:
                cover_start = t
            else:
                if cover_start is not None:
                    cover_periods.append((cover_start, t - DT))
                    cover_start = None

    if cover_start is not None:
        cover_periods.append((cover_start, END_TIME))

    total_cover = 0
    for start, end in cover_periods:
        total_cover += end - start

    print(f"有效遮蔽时长: {total_cover:.4f} 秒")
    print("有效遮蔽时间段:")
    for i, (start, end) in enumerate(cover_periods):
        print(f"  时间段 {i+1}: {start:.4f} 秒 到 {end:.4f} 秒")

    plt.figure(figsize=(10, 5))
    plt.plot(time_pts, cover_states, 'b-', label='遮蔽状态')
    plt.xlabel('时间 (秒)')
    plt.ylabel('遮蔽状态 (1 有效, 0 无效)')

```

```

plt.title('烟雾弹对 M1 的有效遮蔽状态随时间变化')
plt.grid(True)
plt.legend()
plt.savefig('cover_status.png', dpi=300)
plt.show()

fig = plt.figure(figsize=(12, 10))

ax1 = fig.add_subplot(121, projection='3d')

missile_traj = []
for t in time_pts:
    missile_pos = M1_START + missile_vel_vec * t
    missile_traj.append(missile_pos)
missile_traj = np.array(missile_traj)
ax1.plot(missile_traj[:, 0], missile_traj[:, 1], missile_traj[:, 2], 'r-',
        label='导弹轨迹', linewidth=1)

cloud_traj = []
for t in time_pts:
    time_after_burst = t - BURST_TIME
    cloud_pos = np.array([burst_pos[0], burst_pos[1], burst_pos[2] -
        SINK_VEL * time_after_burst])
    cloud_traj.append(cloud_pos)
cloud_traj = np.array(cloud_traj)
ax1.plot(cloud_traj[:, 0], cloud_traj[:, 1], cloud_traj[:, 2], 'g-',
        label='烟雾球轨迹', linewidth=1)

ax1.scatter(samp_pts[:, 0], samp_pts[:, 1], samp_pts[:, 2], c='b',
        marker='.', s=1, alpha=0.5, label='目标样本点')

ax1.scatter(CYL_CENTER[0], CYL_CENTER[1], CYL_CENTER[2], c='k',
        marker='x', s=20, label='圆柱中心')

ax1.scatter(burst_pos[0], burst_pos[1], burst_pos[2], c='m', marker='*',
        s=20, label='起爆点')

ax1.set_xlabel('X')
ax1.set_ylabel('Y')
ax1.set_zlabel('Z')
ax1.set_title('全局视图')
ax1.legend()

ax2 = fig.add_subplot(122, projection='3d')

ax2.scatter(samp_pts[:, 0], samp_pts[:, 1], samp_pts[:, 2], c='b',
        marker='.', s=5, alpha=0.5, label='目标样本点')

ax2.scatter(CYL_CENTER[0], CYL_CENTER[1], CYL_CENTER[2], c='k',
        marker='x', s=20, label='圆柱中心')

ax2.plot(cloud_traj[:, 0], cloud_traj[:, 1], cloud_traj[:, 2], 'g-',
        label='烟雾球轨迹', linewidth=2)

ax2.set_xlim([-20, 20])
ax2.set_ylim([180, 220])
ax2.set_zlim([-5, 15])

ax2.set_xlabel('X')
ax2.set_ylabel('Y')
ax2.set_zlabel('Z')
ax2.set_title('局部视图（放大圆柱区域）')
ax2.legend()

plt.tight_layout()

```

```

plt.savefig('spatial_distribution_final.png', dpi=300)
plt.show()

if __name__ == "__main__":
    main()

```

附录三：

第二小问代码

```

import math
import random
import numpy as np
import matplotlib.pyplot as plt
from numba import jit
import time

plt.rcParams["font.family"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

GRAVITY_ACC = 9.8
MISSILE_VELOCITY = 300
CLOUD_SINK_RATE = 3
EFFECTIVE_RADIUS = 10
EFFECTIVE_DURATION = 20

CYLINDER_CENTER = np.array([0.0, 200.0, 5.0])
CYLINDER_RADIUS = 7.0
CYLINDER_HEIGHT = 10.0

M1_START_POS = np.array([20000.0, 0.0, 2000.0])
DECOY_TARGET_POS = np.array([0.0, 0.0, 0.0])
FY1_START_POS = np.array([17800.0, 0.0, 1800.0])

A_DEFAULT = 52
B_DEFAULT = 12

TIME_STEP = 0.01

missile_dir_vec = (DECOY_TARGET_POS - M1_START_POS)
missile_dir_unit = missile_dir_vec / np.linalg.norm(missile_dir_vec)
missile_velocity_vec = MISSILE_VELOCITY * missile_dir_unit

@jit(nopython=True)
def calc_dist_to_segment(P, A, B):
    vec_AP = P - A
    vec_AB = B - A
    len_AB = np.linalg.norm(vec_AB)
    if len_AB <= 1e-12:
        return np.linalg.norm(P - A)
    dot_val = np.dot(vec_AP, vec_AB)
    if dot_val > 0.0:
        cross_val = np.linalg.norm(np.cross(vec_AP, vec_AB))
        return cross_val / len_AB
    else:
        return np.linalg.norm(P - A)

@jit(nopython=True)
def check_effectiveness_numba(missile_pos, cloud_pos, sample_pts, eff_radius,
cyl_center):
    dist_to_center = calc_dist_to_segment(cloud_pos, missile_pos, cyl_center)

```

```

        if dist_to_center > eff_radius:
            return False
        for i in range(len(sample_pts)):
            pt = sample_pts[i]
            dist_val = calc_dist_to_segment(cloud_pos, missile_pos, pt)
            if dist_val > eff_radius:
                return False
        return True

def generate_cyl_samples(a_num, b_num):
    points_list = []
    for layer_idx in range(b_num + 1):
        z_val = layer_idx * CYLINDER_HEIGHT / b_num
        for i_idx in range(a_num):
            angle_val = 2.0 * math.pi * i_idx / a_num
            x_val = CYLINDER_RADIUS * math.cos(angle_val)
            y_val = 200.0 + CYLINDER_RADIUS * math.sin(angle_val)
            points_list.append([x_val, y_val, z_val])
    return np.array(points_list, dtype=np.float64)

def simulate_and_evaluate(theta_deg, uav_vel, drop_time, burst_delay,
sample_pts, return_segments=False):
    theta_rad = math.radians(theta_deg)
    direction_xy = np.array([math.cos(theta_rad), math.sin(theta_rad), 0.0],
dtype=np.float64)
    velocity_vec = uav_vel * direction_xy

    actual_drop_time = max(0.0, drop_time)
    actual_burst_time = actual_drop_time + max(0.0, burst_delay)

    uav_drop_pos = FY1_START_POS + velocity_vec * actual_drop_time

    horiz_displacement = velocity_vec * burst_delay
    vert_displacement = np.array([0.0, 0.0, -0.5 * GRAVITY_ACC * (burst_delay
** 2)], dtype=np.float64)
    burst_position = uav_drop_pos + horiz_displacement + vert_displacement

    start_time = actual_burst_time
    end_time = actual_burst_time + EFFECTIVE_DURATION

    time_points = np.arange(start_time, end_time + 1e-12, TIME_STEP)
    num_steps = len(time_points)

    cover_intervals = []
    is_covering = False
    segment_start = None

    missile_positions = M1_START_POS + missile_velocity_vec *
time_points.reshape(-1, 1)

    time_after_burst = time_points - actual_burst_time
    cloud_positions = np.empty((num_steps, 3), dtype=np.float64)
    cloud_positions[:, 0] = burst_position[0]
    cloud_positions[:, 1] = burst_position[1]
    cloud_positions[:, 2] = burst_position[2] - CLOUD_SINK_RATE *
time_after_burst

    coverage_status = np.zeros(num_steps, dtype=bool)
    for i in range(num_steps):
        coverage_status[i] = check_effectiveness_numba(
            missile_positions[i],
            cloud_positions[i],
            sample_pts,
            EFFECTIVE_RADIUS,
            CYLINDER_CENTER
        )

```

```

        for i in range(num_steps):
            if coverage_status[i] and not is_covering:
                is_covering = True
                segment_start = time_points[i]
            elif (not coverage_status[i]) and is_covering:
                is_covering = False
                cover_intervals.append((segment_start, time_points[i] -
TIME_STEP))
                segment_start = None

            if is_covering and segment_start is not None:
                cover_intervals.append((segment_start, end_time))

            total_coverage = 0.0
            for start, end in cover_intervals:
                total_coverage += (end - start)

            if return_segments:
                return total_coverage, cover_intervals, uav_drop_pos, burst_position,
actual_drop_time, actual_burst_time
            else:
                return total_coverage

param_bounds = {
    'theta_deg': (0.0, 10.0),
    'uav_vel': (70.0, 140.0),
    'drop_time': (0.0, 55.0),
    'burst_delay': (0.1, 6.0),
}

POPULATION_SIZE = 50
MAX_GENERATIONS = 100
ELITE_COUNT = 2
CROSSOVER_RATE = 0.9
MUTATION_RATE = 0.25
ETA_CROSSOVER = 10.0
ETA_MUTATION = 20.0
RANDOM_SEED = 42

bounds_array = [
    param_bounds['theta_deg'],
    param_bounds['uav_vel'],
    param_bounds['drop_time'],
    param_bounds['burst_delay'],
]

def initialize_population(size):
    population = []
    for _ in range(size):
        individual = np.array([
            np.random.uniform(*param_bounds['theta_deg']),
            np.random.uniform(*param_bounds['uav_vel']),
            np.random.uniform(*param_bounds['drop_time']),
            np.random.uniform(*param_bounds['burst_delay'])
        ], dtype=np.float64)
        population.append(individual)
    return np.array(population, dtype=np.float64)

def constrain_to_bounds(ind):
    ind[0] = np.clip(ind[0], *param_bounds['theta_deg'])
    ind[1] = np.clip(ind[1], *param_bounds['uav_vel'])
    ind[2] = np.clip(ind[2], *param_bounds['drop_time'])
    ind[3] = np.clip(ind[3], *param_bounds['burst_delay'])
    return ind

```



```

def select_by_tournament(fitness_values, k=2):
    idx1 = np.random.randint(0, len(fitness_values))
    idx2 = np.random.randint(0, len(fitness_values))
    return idx1 if fitness_values[idx1] >= fitness_values[idx2] else idx2

def perform_sbx_crossover(parent1, parent2, eta):
    if np.random.rand() > CROSSOVER_RATE:
        return parent1.copy(), parent2.copy()
    child1 = parent1.copy()
    child2 = parent2.copy()
    for i in range(len(parent1)):
        x1, x2 = parent1[i], parent2[i]
        if np.random.rand() < 0.5 and abs(x1 - x2) > 1e-14:
            x_low, x_high = min(x1, x2), max(x1, x2)
            u_val = np.random.rand()
            beta_val = 1.0 + (2.0 * (x_low - bounds_array[i][0]) / (x_high -
x_low))

            alpha_val = 2.0 - (beta_val ** -(eta + 1.0))
            if u_val <= 1.0 / alpha_val:
                beta_q = (u_val * alpha_val) ** (1.0 / (eta + 1.0))
            else:
                beta_q = (1.0 / (2.0 - u_val * alpha_val)) ** (1.0 / (eta +
1.0))

            child1[i] = 0.5 * ((x1 + x2) - beta_q * (x2 - x1))

            u_val = np.random.rand()
            beta_val = 1.0 + (2.0 * (bounds_array[i][1] - x_high) / (x_high -
x_low))

            alpha_val = 2.0 - (beta_val ** -(eta + 1.0))
            if u_val <= 1.0 / alpha_val:
                beta_q = (u_val * alpha_val) ** (1.0 / (eta + 1.0))
            else:
                beta_q = (1.0 / (2.0 - u_val * alpha_val)) ** (1.0 / (eta +
1.0))

            child2[i] = 0.5 * ((x1 + x2) + beta_q * (x2 - x1))
        else:
            child1[i] = x1
            child2[i] = x2
    return constrain_to_bounds(child1), constrain_to_bounds(child2)

def apply_polynomial_mutation(individual, eta):
    for i in range(len(individual)):
        if np.random.rand() < MUTATION_RATE:
            lower_bound, upper_bound = bounds_array[i]
            x_val = individual[i]
            if lower_bound == upper_bound:
                continue
            delta1 = (x_val - lower_bound) / (upper_bound - lower_bound)
            delta2 = (upper_bound - x_val) / (upper_bound - lower_bound)
            u_val = np.random.rand()
            mutation_power = 1.0 / (eta + 1.0)
            if u_val < 0.5:
                xy_val = 1.0 - delta1
                value = 2.0 * u_val + (1.0 - 2.0 * u_val) * (xy_val ** (eta +
1.0))

                delta_q = (value ** mutation_power) - 1.0
            else:
                xy_val = 1.0 - delta2
                value = 2.0 * (1.0 - u_val) + 2.0 * (u_val - 0.5) * (xy_val **
(eta + 1.0))

                delta_q = 1.0 - (value ** mutation_power)
            x_val = x_val + delta_q * (upper_bound - lower_bound)
            individual[i] = np.clip(x_val, lower_bound, upper_bound)
    return individual

def evaluate_fitness(population, sample_pts):

```

```

        fitness_scores = np.zeros(len(population), dtype=np.float64)
        for idx, ind in enumerate(population):
            theta_deg, uav_vel, drop_time, burst_delay = ind
            total_cover = simulate_and_evaluate(theta_deg, uav_vel, drop_time,
burst_delay, sample_pts)
            fitness_scores[idx] = total_cover**1.2
        return fitness_scores

def run_genetic_algorithm():
    samples = generate_cyl_samples(A_DEFAULT, B_DEFAULT)
    print(f"样本点数量: {len(samples)}")

    population = initialize_population(POPULATION_SIZE)
    fitness_scores = evaluate_fitness(population, samples)

    best_fitness_history = []
    best_individual_history = []

    best_index = int(np.argmax(fitness_scores))
    best_fitness = float(fitness_scores[best_index])
    best_individual = population[best_index].copy()
    best_fitness_history.append(best_fitness)
    best_individual_history.append(best_individual.copy())

    stall_count = 0
    STALL_THRESHOLD = 30

    start_time = time.time()

    for generation in range(1, MAX_GENERATIONS + 1):
        new_population = []

        elite_indices = fitness_scores.argsort()[-ELITE_COUNT:][::-1]
        elites = [population[i].copy() for i in elite_indices]
        new_population.extend(elites)

        while len(new_population) < POPULATION_SIZE:
            parent1_idx = select_by_tournament(fitness_scores, k=2)
            parent2_idx = select_by_tournament(fitness_scores, k=2)
            parent1 = population[parent1_idx].copy()
            parent2 = population[parent2_idx].copy()

            child1, child2 = perform_sbx_crossover(parent1, parent2,
ETA_CROSSOVER)
            child1 = apply_polynomial_mutation(child1, ETA_MUTATION)
            child2 = apply_polynomial_mutation(child2, ETA_MUTATION)

            new_population.append(child1)
            if len(new_population) < POPULATION_SIZE:
                new_population.append(child2)

        population = np.array(new_population, dtype=np.float64)
        fitness_scores = evaluate_fitness(population, samples)

        current_best_index = int(np.argmax(fitness_scores))
        current_best_fitness = float(fitness_scores[current_best_index])
        current_best_individual = population[current_best_index].copy()

        if current_best_fitness > best_fitness + 1e-9:
            best_fitness = current_best_fitness
            best_individual = current_best_individual.copy()
            stall_count = 0
        else:
            stall_count += 1

        best_fitness_history.append(best_fitness)

```

```

        best_individual_history.append(best_individual.copy())

        if best_fitness != 0 and stall_count >= STALL_THRESHOLD:
            print(f"[Early Stop] 连续 {STALL_THRESHOLD} 代未提升，在第
{generation} 代停止。")
            break

        if generation % 5 == 0 or generation == 1:
            elapsed = time.time() - start_time
            print(f"代 {generation:3d} | 代内最佳={current_best_fitness:.4f} |
历史最佳={best_fitness:.4f} | 耗时: {elapsed:.2f}s")

    return best_individual, best_fitness, np.array(best_fitness_history),
samples

def main():
    print("开始遗传算法优化...")
    best_individual, best_fitness, fitness_curve, samples =
run_genetic_algorithm()
    theta_deg, uav_vel, drop_time, burst_delay = best_individual.tolist()

    print("对最优解进行精细评估...")
    total_cover, segments, drop_pos, burst_pos, actual_drop_time,
actual_burst_time = simulate_and_evaluate(
        theta_deg, uav_vel, drop_time, burst_delay, samples,
return_segments=True
    )

    print("\n===== 最优解（历史种群） =====")
    print(f"相对 x 轴正向的飞行倾角  $\theta$  (度) : {theta_deg:.4f}")
    print(f"无人机巡航速度 v (m/s) : {uav_vel:.4f}")
    print(f"投放延迟 t_drop (s) : {drop_time:.4f}")
    print(f"起爆延迟  $\Delta t_{burst}$  (s) : {burst_delay:.4f}")

    print("\n----- 由最优解推得的位置 -----")
    print(f"烟幕干扰弹投放点 (x, y, z) : ({drop_pos[0]:.3f},
{drop_pos[1]:.3f}, {drop_pos[2]:.3f})")
    print(f"烟幕干扰弹起爆点 (x, y, z) : ({burst_pos[0]:.3f},
{burst_pos[1]:.3f}, {burst_pos[2]:.3f})")

    print("\n----- 遮蔽评估（双判定） -----")
    print(f"最长'有效遮挡'总时长 (s) : {total_cover:.4f}")
    if len(segments) == 0:
        print("无有效遮挡时间段。")
    else:
        print("有效遮挡时间段: ")
        for k, (start, end) in enumerate(segments, 1):
            print(f" 段 {k:02d}: {start:.3f} s ~ {end:.3f} s (持续 {end -
start:.3f} s)")

    plt.figure(figsize=(7.2, 4.8))
    plt.plot(fitness_curve, lw=2)
    plt.xlabel("迭代代数")
    plt.ylabel("历史最佳适应度(有效遮挡时长,s)")
    plt.title("适应度随迭代次数的变化(历史最佳)")
    plt.grid(True, alpha=0.4)
    plt.tight_layout()
    plt.savefig("Q2_fitness_curve.png", dpi=160)
    print("\n 已保存迭代曲线: Q2_fitness_curve.png")

if __name__ == "__main__":
    main()

```

附录四：

第三小问代码 1 (用于优化求解最优投放方案)

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from numba import jit
import time
import math

plt.rcParams["font.family"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

GRAVITY = 9.8
MISSILE_VEL = 300
CLOUD_SINK_RATE = 3
EFFECTIVE_RADIUS = 10
EFFECTIVE_DURATION = 20

CYLINDER_CENTER = np.array([0.0, 200.0, 5.0])
CYLINDER_RADIUS = 7.0
CYLINDER_HEIGHT = 10.0

M1_INIT = np.array([20000.0, 0.0, 2000.0])
DECOY_TARGET = np.array([0.0, 0.0, 0.0])
FY1_INIT = np.array([17800.0, 0.0, 1800.0])

A_OPT = 52
B_OPT = 12

TIME_STEP = 0.01

missile_dir = (DECOY_TARGET - M1_INIT)
missile_dir_unit = missile_dir / np.linalg.norm(missile_dir)
missile_vel_vec = MISSILE_VEL * missile_dir_unit

@jit(nopython=True)
def calc_segment_dist(P, A, B):
    AP = P - A
    AB = B - A
    ab_len = np.linalg.norm(AB)
    if ab_len <= 1e-12:
        return np.linalg.norm(P - A)
    dot_val = np.dot(AP, AB)
    if dot_val > 0.0:
        cross_val = np.linalg.norm(np.cross(AP, AB))
        return cross_val / ab_len
    else:
        return np.linalg.norm(P - A)

@jit(nopython=True)
def check_coverage_batch(missile_positions, smoke_positions, sample_points,
n_smokes):
    n_times = len(missile_positions)
    n_samples = len(sample_points)
    coverage_status = np.zeros(n_times, dtype=np.bool_)

    for t_idx in range(n_times):
        missile_pos = missile_positions[t_idx]
        all_covered = True

        for s_idx in range(n_samples):
            sample_pt = sample_points[s_idx]
```

```

        sample_covered = False

        for smoke_idx in range(n_smokes):
            smoke_pos = smoke_positions[smoke_idx, t_idx]
            if smoke_pos[2] > -999:
                dist_val = calc_segment_dist(smoke_pos, missile_pos,
sample_pt)

                if dist_val < EFFECTIVE_RADIUS:
                    sample_covered = True
                    break

        if not sample_covered:
            all_covered = False
            break

        coverage_status[t_idx] = all_covered

    return coverage_status

def generate_cylinder_samples(a_num, b_num):
    points_list = []
    for layer in range(b_num + 1):
        z_val = layer * CYLINDER_HEIGHT / b_num
        for i in range(a_num):
            angle_val = 2.0 * math.pi * i / a_num
            x_val = CYLINDER_RADIUS * math.cos(angle_val)
            y_val = 200.0 + CYLINDER_RADIUS * math.sin(angle_val)
            points_list.append([x_val, y_val, z_val])
    return np.array(points_list, dtype=np.float64)

def compute_smoke_trajectories(burst_positions, burst_times, time_points):
    n_smokes = len(burst_positions)
    n_times = len(time_points)

    smoke_positions = np.full((n_smokes, n_times, 3), -1000.0,
dtype=np.float64)

    for i in range(n_smokes):
        burst_pos = burst_positions[i]
        burst_time = burst_times[i]

        time_after_burst = time_points - burst_time
        valid_mask = (time_after_burst >= 0) & (time_after_burst <=
EFFECTIVE_DURATION)

        smoke_positions[i, valid_mask, 0] = burst_pos[0]
        smoke_positions[i, valid_mask, 1] = burst_pos[1]
        smoke_positions[i, valid_mask, 2] = burst_pos[2] - CLOUD_SINK_RATE *
time_after_burst[valid_mask]

    return smoke_positions

def simulate_three_smokes(theta_deg, uav_vel, drop_times, burst_delays,
sample_points, return_segments=False):
    theta_rad = math.radians(theta_deg)
    dir_xy = np.array([math.cos(theta_rad), math.sin(theta_rad), 0.0],
dtype=np.float64)
    vel_vec = uav_vel * dir_xy

    burst_times_list = []
    burst_positions_list = []
    drop_positions_list = []

    for i in range(3):
        t_drop = drop_times[i]
        t_burst_delay = burst_delays[i]

```

```

        actual_drop_time = max(0.0, t_drop)
        actual_burst_time = actual_drop_time + max(0.0, t_burst_delay)

        uav_drop_pos = FY1_INIT + vel_vec * actual_drop_time
        drop_positions_list.append(uav_drop_pos)

        horiz_disp = vel_vec * t_burst_delay
        vert_disp = np.array([0.0, 0.0, -0.5 * GRAVITY * (t_burst_delay **
2)], dtype=np.float64)
        burst_pos = uav_drop_pos + horiz_disp + vert_disp

        burst_times_list.append(actual_burst_time)
        burst_positions_list.append(burst_pos)

    start_time = min(burst_times_list)
    end_time = max(burst_times_list) + EFFECTIVE_DURATION

    time_points_arr = np.arange(start_time, end_time + 1e-12, TIME_STEP)
    n_steps = len(time_points_arr)

    missile_positions = M1_INIT + missile_vel_vec * time_points_arr.reshape(-
1, 1)

    smoke_trajectories =
compute_smoke_trajectories(np.array(burst_positions_list),
np.array(burst_times_list), time_points_arr)

    coverage_status = check_coverage_batch(missile_positions,
smoke_trajectories, sample_points, 3)

    cover_segments = []
    is_covering = False
    seg_start = None

    for i in range(n_steps):
        if coverage_status[i] and not is_covering:
            is_covering = True
            seg_start = time_points_arr[i]
        elif (not coverage_status[i]) and is_covering:
            is_covering = False
            cover_segments.append((seg_start, time_points_arr[i] - TIME_STEP))
            seg_start = None

    if is_covering and seg_start is not None:
        cover_segments.append((seg_start, end_time))

    total_cover = sum(e - s for s, e in cover_segments)

    if return_segments:
        return total_cover, cover_segments, drop_positions_list,
burst_positions_list, burst_times_list, time_points_arr, coverage_status
    else:
        return total_cover

param_ranges = {
    'theta_deg': (150.0, 180.0),
    'uav_vel': (70.0, 140.0),
    'drop_time1': (0.0, 10.0),
    'drop_time2': (1.0, 15.0),
    'drop_time3': (2.0, 20.0),
    'burst_delay1': (0.1, 6.0),
    'burst_delay2': (0.1, 6.0),
    'burst_delay3': (0.1, 6.0),
}

```

```

POP_SIZE = 50
MAX_GEN = 120
F0 = 0.7
CR = 0.8

def adaptive_scale_factor(gen, max_gen, base_f):
    return base_f * (0.4 + 0.6 * (1 - gen / max_gen))

def initialize_population(size):
    population = []
    for _ in range(size):
        theta_deg = np.random.uniform(*param_ranges['theta_deg'])
        uav_vel = np.random.uniform(*param_ranges['uav_vel'])

        t_drop1 = np.random.uniform(*param_ranges['drop_time1'])
        t_drop2 = np.random.uniform(max(param_ranges['drop_time2'][0],
t_drop1 + 1.0), param_ranges['drop_time2'][1])
        t_drop3 = np.random.uniform(max(param_ranges['drop_time3'][0],
t_drop2 + 1.0), param_ranges['drop_time3'][1])

        t_burst_delay1 = np.random.uniform(*param_ranges['burst_delay1'])
        t_burst_delay2 = np.random.uniform(*param_ranges['burst_delay2'])
        t_burst_delay3 = np.random.uniform(*param_ranges['burst_delay3'])

        individual = np.array([theta_deg, uav_vel, t_drop1, t_drop2, t_drop3,
                                t_burst_delay1, t_burst_delay2, t_burst_delay3],
dtype=np.float64)
        population.append(individual)
    return np.array(population, dtype=np.float64)

def repair_individual(ind):
    t_drop1, t_drop2, t_drop3 = ind[2], ind[3], ind[4]

    t_drop1 = np.clip(t_drop1, *param_ranges['drop_time1'])
    t_drop2 = max(t_drop1 + 1.0, np.clip(t_drop2,
*param_ranges['drop_time2']))
    t_drop3 = max(t_drop2 + 1.0, np.clip(t_drop3,
*param_ranges['drop_time3']))

    ind[0] = np.clip(ind[0], *param_ranges['theta_deg'])
    ind[1] = np.clip(ind[1], *param_ranges['uav_vel'])
    ind[2] = t_drop1
    ind[3] = t_drop2
    ind[4] = t_drop3
    ind[5] = np.clip(ind[5], *param_ranges['burst_delay1'])
    ind[6] = np.clip(ind[6], *param_ranges['burst_delay2'])
    ind[7] = np.clip(ind[7], *param_ranges['burst_delay3'])

    return ind

def run_de_optimization(sample_points):
    print(f"开始差分进化优化...")
    print(f"种群大小: {POP_SIZE}, 最大代数: {MAX_GEN}")
    print(f"样本点数量: {len(sample_points)}")

    population = initialize_population(POP_SIZE)

    fitness = np.zeros(POP_SIZE)
    for i in range(POP_SIZE):
        ind = population[i]
        theta_deg, uav_vel = ind[0], ind[1]
        drop_times = [ind[2], ind[3], ind[4]]
        burst_delays = [ind[5], ind[6], ind[7]]

        fitness[i] = simulate_three_smokes(theta_deg, uav_vel, drop_times,
burst_delays, sample_points)

```

```

best_fit_history = []

best_idx = np.argmax(fitness)
best_fit = fitness[best_idx]
best_ind = population[best_idx].copy()

best_fit_history.append(best_fit)
print(f"初始最佳适应度: {best_fit:.4f}秒")

for gen in range(1, MAX_GEN + 1):
    F = adaptive_scale_factor(gen, MAX_GEN, F0)

    new_population = population.copy()
    new_fitness = fitness.copy()

    for i in range(POP_SIZE):
        indices = [idx for idx in range(POP_SIZE) if idx != i]
        a, b, c = population[np.random.choice(indices, 3, replace=False)]

        mutant = a + F * (b - c)
        mutant = repair_individual(mutant)

        trial = population[i].copy()
        j_rand = np.random.randint(0, 8)

        for j in range(8):
            if np.random.rand() < CR or j == j_rand:
                trial[j] = mutant[j]

        trial = repair_individual(trial)

        theta_deg, uav_vel = trial[0], trial[1]
        drop_times = [trial[2], trial[3], trial[4]]
        burst_delays = [trial[5], trial[6], trial[7]]

        trial_fitness = simulate_three_smokes(theta_deg, uav_vel,
        drop_times, burst_delays, sample_points)

        if trial_fitness > fitness[i]:
            new_population[i] = trial
            new_fitness[i] = trial_fitness

            if trial_fitness > best_fit:
                best_fit = trial_fitness
                best_ind = trial.copy()

    population = new_population
    fitness = new_fitness
    best_fit_history.append(best_fit)

    if gen % 5 == 0 or gen == 1:
        print(f"代 {gen:3d} | 最佳适应度: {best_fit:.4f}秒 | 缩放因子 F:
{F:.3f}")

    return best_ind, best_fit, np.array(best_fit_history)

def main():
    print("="*60)
    print("三枚烟幕弹投放策略优化")
    print("="*60)

    samples = generate_cylinder_samples(A_OPT, B_OPT)
    print(f"圆柱采样点数量: {len(samples)} (优化后: {A_OPT}*{B_OPT+1})")
    print(f"时间步长: {TIME_STEP}秒")

```



```

start_time = time.time()
best_ind, best_fit, best_curve = run_de_optimization(samples)
end_time = time.time()

print(f"\n 优化完成! 总耗时: {end_time - start_time:.2f}秒")
print(f"最佳适应度(总遮蔽时长): {best_fit:.4f}秒")

theta_deg, uav_vel = best_ind[0], best_ind[1]
t_drops = [best_ind[2], best_ind[3], best_ind[4]]
t_burst_delays = [best_ind[5], best_ind[6], best_ind[7]]

print("\n" + "="*50)
print("最优解参数")
print("="*50)
print(f"无人机飞行方向  $\theta$ (度): {theta_deg:.4f}")
print(f"无人机飞行速度 v(m/s): {uav_vel:.4f}")

total_cover, segments, drop_positions, burst_positions, burst_times,
time_points, cover_status = simulate_three_smokes(
    theta_deg, uav_vel, t_drops, t_burst_delays, samples,
    return_segments=True
)

single_cover_statuses = []
single_time_points_list = []

for i in range(3):
    single_cover, _, _, _, _, single_time_points, single_cover_status
= simulate_three_smokes(
    theta_deg, uav_vel, [t_drops[i]], [t_burst_delays[i]], samples,
    return_segments=True
)
    single_cover_statuses.append(single_cover_status)
    single_time_points_list.append(single_time_points)

smoke_details = []
for i in range(3):
    smoke_details.append({
        'drop_pos': drop_positions[i],
        'burst_pos': burst_positions[i],
        'drop_time': t_drops[i],
        'burst_time': burst_times[i],
        'cover_time': 0.0
    })

    single_cover = simulate_three_smokes(
        theta_deg, uav_vel, [t_drops[i]], [t_burst_delays[i]], samples,
        return_segments=False
    )
    smoke_details[i]['cover_time'] = single_cover

    print(f"\n 烟幕弹 {i+1}:")
    print(f"  投放时间: {t_drops[i]:.4f}s")
    print(f"  起爆延迟: {t_burst_delays[i]:.4f}s")
    print(f"  投放点: ({drop_positions[i][0]:.3f},
{drop_positions[i][1]:.3f}, {drop_positions[i][2]:.3f})")
    print(f"  起爆点: ({burst_positions[i][0]:.3f},
{burst_positions[i][1]:.3f}, {burst_positions[i][2]:.3f})")
    print(f"  单独遮蔽时长: {single_cover:.4f}s")

plt.figure(figsize=(12, 8))

joint_cover_numeric = np.where(cover_status, 2, 0)

```

```

plt.plot(time_points, joint_cover_numeric, 'r-', linewidth=2.5, label='联合遮蔽状态 (2=有效, 0=无效)')

colors = ['b', 'g', 'm']
labels = ['烟雾弹 1 单独遮蔽状态 (1=有效, 0=无效)',
          '烟雾弹 2 单独遮蔽状态 (1=有效, 0=无效)',
          '烟雾弹 3 单独遮蔽状态 (1=有效, 0=无效)']

for i in range(3):
    single_cover_numeric = np.where(single_cover_statuses[i], 1, 0)
    plt.plot(single_time_points_list[i], single_cover_numeric,
             color=colors[i], linestyle='--', linewidth=1.5,
label=labels[i])

plt.xlabel('时间 (秒)')
plt.ylabel('遮蔽状态')
plt.title('烟雾弹单独和联合遮蔽状态随时间变化')
plt.grid(True, alpha=0.3)
plt.legend(loc='upper right')
plt.tight_layout()
plt.savefig("cover_status_comparison.png", dpi=160)
print("\n 已保存遮蔽状态对比图: cover_status_comparison.png")

plt.figure(figsize=(10, 6))
plt.plot(best_curve, lw=2, color='blue')
plt.xlabel("迭代代数")
plt.ylabel("历史最佳适应度(总遮蔽时长, s)")
plt.title("三枚烟雾弹优化适应度曲线")
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig("fitness_curve_3smokes.png", dpi=160)
print("已保存适应度曲线: fitness_curve_3smokes.png")

result_data = []
for i in range(3):
    detail = smoke_details[i]
    result_data.append({
        '无人机运动方向': theta_deg,
        '无人机运动速度 (m/s)': uav_vel,
        '烟雾干扰弹编号': i+1,
        '烟雾干扰弹投放点的 x 坐标 (m)': detail['drop_pos'][0],
        '烟雾干扰弹投放点的 y 坐标 (m)': detail['drop_pos'][1],
        '烟雾干扰弹投放点的 z 坐标 (m)': detail['drop_pos'][2],
        '烟雾干扰弹起爆点的 x 坐标 (m)': detail['burst_pos'][0],
        '烟雾干扰弹起爆点的 y 坐标 (m)': detail['burst_pos'][1],
        '烟雾干扰弹起爆点的 z 坐标 (m)': detail['burst_pos'][2],
        '有效干扰时长 (s)': detail['cover_time']
    })

df = pd.DataFrame(result_data)
df.to_csv('result1.csv', index=False, encoding='utf-8-sig')
print("\n 已保存结果到: result1.csv")

print("\n 结果表格:")
print(df.to_string(index=False))

print(f"\n 总的联合遮蔽时长: {best_fit:.4f}秒")
print("优化完成!")

if __name__ == "__main__":
    main()

```

附录五：

第三小问代码 2 (用于对第三小问中的算法进行灵敏度分析)

```
import numpy as np
import matplotlib.pyplot as plt
from numba import jit
import math
import time

plt.rcParams["font.family"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

GRAVITY = 9.8
MISSILE_VEL = 300
CLOUD_SINK_RATE = 3
EFFECTIVE_RADIUS = 10
EFFECTIVE_DURATION = 20

CYLINDER_CENTER = np.array([0.0, 200.0, 5.0])
CYLINDER_RADIUS = 7.0
CYLINDER_HEIGHT = 10.0

M1_INIT = np.array([20000.0, 0.0, 2000.0])
DECOY_TARGET = np.array([0.0, 0.0, 0.0])
FY1_INIT = np.array([17800.0, 0.0, 1800.0])

A_OPT = 36
B_OPT = 10

TIME_STEP = 0.02

missile_dir = (DECOY_TARGET - M1_INIT)
missile_dir_unit = missile_dir / np.linalg.norm(missile_dir)
missile_vel_vec = MISSILE_VEL * missile_dir_unit

@jit(nopython=True)
def calc_segment_dist(P, A, B):
    AP = P - A
    AB = B - A
    ab_len = np.linalg.norm(AB)
    if ab_len <= 1e-12:
        return np.linalg.norm(P - A)
    dot_val = np.dot(AP, AB)
    if dot_val > 0.0:
        cross_val = np.linalg.norm(np.cross(AP, AB))
        return cross_val / ab_len
    else:
        return np.linalg.norm(P - A)

@jit(nopython=True)
def check_coverage_batch(missile_positions, smoke_positions, sample_points,
n_smokes):
    n_times = len(missile_positions)
    n_samples = len(sample_points)
    coverage_status = np.zeros(n_times, dtype=np.bool_)

    for t_idx in range(n_times):
        missile_pos = missile_positions[t_idx]
        all_covered = True

        for s_idx in range(n_samples):
            sample_pt = sample_points[s_idx]
```

```

        sample_covered = False

        for smoke_idx in range(n_smokes):
            smoke_pos = smoke_positions[smoke_idx, t_idx]
            if smoke_pos[2] > -999:
                dist_val = calc_segment_dist(smoke_pos, missile_pos,
sample_pt)

                if dist_val < EFFECTIVE_RADIUS:
                    sample_covered = True
                    break

        if not sample_covered:
            all_covered = False
            break

        coverage_status[t_idx] = all_covered

    return coverage_status

def generate_cylinder_samples(a_num, b_num):
    points_list = []
    for layer in range(b_num + 1):
        z_val = layer * CYLINDER_HEIGHT / b_num
        for i in range(a_num):
            angle_val = 2.0 * math.pi * i / a_num
            x_val = CYLINDER_RADIUS * math.cos(angle_val)
            y_val = 200.0 + CYLINDER_RADIUS * math.sin(angle_val)
            points_list.append([x_val, y_val, z_val])
    return np.array(points_list, dtype=np.float64)

def compute_smoke_trajectories(burst_positions, burst_times, time_points):
    n_smokes = len(burst_positions)
    n_times = len(time_points)

    smoke_positions = np.full((n_smokes, n_times, 3), -1000.0,
dtype=np.float64)

    for i in range(n_smokes):
        burst_pos = burst_positions[i]
        burst_time = burst_times[i]

        time_after_burst = time_points - burst_time
        valid_mask = (time_after_burst >= 0) & (time_after_burst <=
EFFECTIVE_DURATION)

        smoke_positions[i, valid_mask, 0] = burst_pos[0]
        smoke_positions[i, valid_mask, 1] = burst_pos[1]
        smoke_positions[i, valid_mask, 2] = burst_pos[2] - CLOUD_SINK_RATE *
time_after_burst[valid_mask]

    return smoke_positions

def simulate_three_smokes(theta_deg, uav_vel, drop_times, burst_delays,
sample_points, return_segments=False):
    theta_rad = math.radians(theta_deg)
    dir_xy = np.array([math.cos(theta_rad), math.sin(theta_rad), 0.0],
dtype=np.float64)
    vel_vec = uav_vel * dir_xy

    burst_times_list = []
    burst_positions_list = []
    drop_positions_list = []

    for i in range(3):
        t_drop = drop_times[i]
        t_burst_delay = burst_delays[i]

```

```

        actual_drop_time = max(0.0, t_drop)
        actual_burst_time = actual_drop_time + max(0.0, t_burst_delay)

        uav_drop_pos = FY1_INIT + vel_vec * actual_drop_time
        drop_positions_list.append(uav_drop_pos)

        horiz_disp = vel_vec * t_burst_delay
        vert_disp = np.array([0.0, 0.0, -0.5 * GRAVITY * (t_burst_delay **
2)], dtype=np.float64)
        burst_pos = uav_drop_pos + horiz_disp + vert_disp

        burst_times_list.append(actual_burst_time)
        burst_positions_list.append(burst_pos)

    start_time = min(burst_times_list)
    end_time = max(burst_times_list) + EFFECTIVE_DURATION

    time_points_arr = np.arange(start_time, end_time + 1e-12, TIME_STEP)
    n_steps = len(time_points_arr)

    missile_positions = M1_INIT + missile_vel_vec * time_points_arr.reshape(-
1, 1)

    smoke_trajectories =
compute_smoke_trajectories(np.array(burst_positions_list),
np.array(burst_times_list), time_points_arr)

    coverage_status = check_coverage_batch(missile_positions,
smoke_trajectories, sample_points, 3)

    cover_segments = []
    is_covering = False
    seg_start = None

    for i in range(n_steps):
        if coverage_status[i] and not is_covering:
            is_covering = True
            seg_start = time_points_arr[i]
        elif (not coverage_status[i]) and is_covering:
            is_covering = False
            cover_segments.append((seg_start, time_points_arr[i] - TIME_STEP))
            seg_start = None

    if is_covering and seg_start is not None:
        cover_segments.append((seg_start, end_time))

    total_cover = sum(e - s for s, e in cover_segments)

    if return_segments:
        return total_cover, cover_segments, drop_positions_list,
burst_positions_list, burst_times_list, time_points_arr, coverage_status
    else:
        return total_cover

param_ranges = {
    'theta_deg': (0.0, 180.0),
    'uav_vel': (70.0, 140.0),
    'drop_time1': (0.0, 10.0),
    'drop_time2': (1.0, 15.0),
    'drop_time3': (2.0, 20.0),
    'burst_delay1': (0.1, 6.0),
    'burst_delay2': (0.1, 6.0),
    'burst_delay3': (0.1, 6.0),
}

```

```

POP_SIZE_DEF = 50
MAX_GEN_DEF = 70
F0_DEF = 0.7
CR_DEF = 0.8

def adaptive_scale_factor(gen, max_gen, base_f):
    return base_f * (0.4 + 0.6 * (1 - gen / max_gen))

def initialize_population(size):
    population = []
    for _ in range(size):
        theta_deg = np.random.uniform(*param_ranges['theta_deg'])
        uav_vel = np.random.uniform(*param_ranges['uav_vel'])

        t_drop1 = np.random.uniform(*param_ranges['drop_time1'])
        t_drop2 = np.random.uniform(max(param_ranges['drop_time2'][0],
t_drop1 + 1.0), param_ranges['drop_time2'][1])
        t_drop3 = np.random.uniform(max(param_ranges['drop_time3'][0],
t_drop2 + 1.0), param_ranges['drop_time3'][1])

        t_burst_delay1 = np.random.uniform(*param_ranges['burst_delay1'])
        t_burst_delay2 = np.random.uniform(*param_ranges['burst_delay2'])
        t_burst_delay3 = np.random.uniform(*param_ranges['burst_delay3'])

        individual = np.array([theta_deg, uav_vel, t_drop1, t_drop2, t_drop3,
t_burst_delay1, t_burst_delay2, t_burst_delay3],
dtype=np.float64)
        population.append(individual)
    return np.array(population, dtype=np.float64)

def repair_individual(ind):
    t_drop1, t_drop2, t_drop3 = ind[2], ind[3], ind[4]

    t_drop1 = np.clip(t_drop1, *param_ranges['drop_time1'])
    t_drop2 = max(t_drop1 + 1.0, np.clip(t_drop2,
*param_ranges['drop_time2']))
    t_drop3 = max(t_drop2 + 1.0, np.clip(t_drop3,
*param_ranges['drop_time3']))

    ind[0] = np.clip(ind[0], *param_ranges['theta_deg'])
    ind[1] = np.clip(ind[1], *param_ranges['uav_vel'])
    ind[2] = t_drop1
    ind[3] = t_drop2
    ind[4] = t_drop3
    ind[5] = np.clip(ind[5], *param_ranges['burst_delay1'])
    ind[6] = np.clip(ind[6], *param_ranges['burst_delay2'])
    ind[7] = np.clip(ind[7], *param_ranges['burst_delay3'])

    return ind

def run_de_optimization(sample_points, pop_size, max_gen, f0, cr):
    print(f"开始差分进化优化...")
    print(f"种群大小: {pop_size}, 最大代数: {max_gen}, F0: {f0}, CR: {cr}")
    print(f"样本点数量: {len(sample_points)}")

    population = initialize_population(pop_size)

    fitness = np.zeros(pop_size)
    for i in range(pop_size):
        ind = population[i]
        theta_deg, uav_vel = ind[0], ind[1]
        drop_times = [ind[2], ind[3], ind[4]]
        burst_delays = [ind[5], ind[6], ind[7]]

        fitness[i] = simulate_three_smokes(theta_deg, uav_vel, drop_times,
burst_delays, sample_points)

```

```

best_fit_history = []

best_idx = np.argmax(fitness)
best_fit = fitness[best_idx]
best_ind = population[best_idx].copy()

best_fit_history.append(best_fit)
print(f"初始最佳适应度: {best_fit:.4f}秒")

for gen in range(1, max_gen + 1):
    F = adaptive_scale_factor(gen, max_gen, f0)

    new_population = population.copy()
    new_fitness = fitness.copy()

    for i in range(pop_size):
        indices = [idx for idx in range(pop_size) if idx != i]
        a, b, c = population[np.random.choice(indices, 3, replace=False)]

        mutant = a + F * (b - c)
        mutant = repair_individual(mutant)

        trial = population[i].copy()
        j_rand = np.random.randint(0, 8)

        for j in range(8):
            if np.random.rand() < cr or j == j_rand:
                trial[j] = mutant[j]

        trial = repair_individual(trial)

        theta_deg, uav_vel = trial[0], trial[1]
        drop_times = [trial[2], trial[3], trial[4]]
        burst_delays = [trial[5], trial[6], trial[7]]

        trial_fitness = simulate_three_smokes(theta_deg, uav_vel,
        drop_times, burst_delays, sample_points)

        if trial_fitness > fitness[i]:
            new_population[i] = trial
            new_fitness[i] = trial_fitness

            if trial_fitness > best_fit:
                best_fit = trial_fitness
                best_ind = trial.copy()

    population = new_population
    fitness = new_fitness
    best_fit_history.append(best_fit)

    if gen % 5 == 0 or gen == 1:
        print(f"代 {gen:3d} | 最佳适应度: {best_fit:.4f}秒 | 缩放因子 F:
        {F:.3f}")

    return best_ind, best_fit, np.array(best_fit_history)

def perform_sensitivity_analysis():
    samples = generate_cylinder_samples(A_OPT, B_OPT)
    print(f"圆柱采样点数量: {len(samples)}")

    default_pop_size = 50
    default_max_gen = 70
    default_f0 = 0.7
    default_cr = 0.8

```

```

f0_values = [0.1, 0.3, 0.5, 0.7, 0.9]
cr_values = [0.1, 0.3, 0.5, 0.7, 0.9]

n_runs = 1

results_f0 = []
results_cr = []

print("正在进行 F0 的灵敏度分析...")
for f0 in f0_values:
    fitness_values = []
    for run in range(n_runs):
        _, best_fit, _ = run_de_optimization(samples, default_pop_size,
default_max_gen, f0, default_cr)
        fitness_values.append(best_fit)
    avg_fitness = np.mean(fitness_values)
    results_f0.append(avg_fitness)
    print(f"F0 = {f0}, 平均最佳适应度 = {avg_fitness:.4f}")

print("正在进行 CR 的灵敏度分析...")
for cr in cr_values:
    fitness_values = []
    for run in range(n_runs):
        _, best_fit, _ = run_de_optimization(samples, default_pop_size,
default_max_gen, default_f0, cr)
        fitness_values.append(best_fit)
    avg_fitness = np.mean(fitness_values)
    results_cr.append(avg_fitness)
    print(f"CR = {cr}, 平均最佳适应度 = {avg_fitness:.4f}")

plt.figure(figsize=(10, 6))
plt.plot(f0_values, results_f0, color='green', linestyle='-.',
linewidth=2.5, alpha=0.8, label='F0')
plt.plot(cr_values, results_cr, color='purple', linestyle=':',
linewidth=2.5, alpha=0.8, label='CR')

plt.xlabel('参数值')
plt.ylabel('平均最佳适应度(有效遮蔽总时长/s)')
plt.title('差分进化算法参数灵敏度分析')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('sensitivity_analysis.png', dpi=300)
plt.show()

print("灵敏度分析完成, 结果已保存到 sensitivity_analysis.png")

if __name__ == '__main__':
    perform_sensitivity_analysis()

```

附录六:

第四小问代码

```

import math
import random
import numpy as np
import matplotlib.pyplot as plt
from numba import jit
import time

```



```

plt.rcParams["font.family"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

GRAVITY_ACC = 9.8
MISSILE_VEL = 300
CLOUD_SINK_RATE = 3
EFFECTIVE_RADIUS = 10
EFFECTIVE_DURATION = 20

CYLINDER_CENTER = np.array([0.0, 200.0, 5.0])
CYLINDER_RADIUS = 7.0
CYLINDER_HEIGHT = 10.0

M1_START_POS = np.array([20000.0, 0.0, 2000.0])
DECOY_TARGET_POS = np.array([0.0, 0.0, 0.0])
FY2_START_POS = np.array([12000.0, 1400.0, 1400.0])

A_DEFAULT = 52
B_DEFAULT = 12

TIME_STEP = 0.01

missile_dir_vec = (DECOY_TARGET_POS - M1_START_POS)
missile_dir_unit = missile_dir_vec / np.linalg.norm(missile_dir_vec)
missile_velocity_vec = MISSILE_VEL * missile_dir_unit

@jit(nopython=True)
def calc_dist_to_segment(P, A, B):
    vec_AP = P - A
    vec_AB = B - A
    len_AB = np.linalg.norm(vec_AB)
    if len_AB <= 1e-12:
        return np.linalg.norm(P - A)
    dot_val = np.dot(vec_AP, vec_AB)
    if dot_val > 0.0:
        cross_val = np.linalg.norm(np.cross(vec_AP, vec_AB))
        return cross_val / len_AB
    else:
        return np.linalg.norm(P - A)

@jit(nopython=True)
def check_effectiveness_numba(missile_pos, cloud_pos, sample_pts, eff_radius,
cyl_center):
    dist_to_center = calc_dist_to_segment(cloud_pos, missile_pos, cyl_center)
    if dist_to_center > eff_radius:
        return False
    for i in range(len(sample_pts)):
        pt = sample_pts[i]
        dist_val = calc_dist_to_segment(cloud_pos, missile_pos, pt)
        if dist_val > eff_radius:
            return False
    return True

def generate_cyl_samples(a_num, b_num):
    points_list = []
    for layer_idx in range(b_num + 1):
        z_val = layer_idx * CYLINDER_HEIGHT / b_num
        for i_idx in range(a_num):
            angle_val = 2.0 * math.pi * i_idx / a_num
            x_val = CYLINDER_RADIUS * math.cos(angle_val)
            y_val = 200.0 + CYLINDER_RADIUS * math.sin(angle_val)
            points_list.append([x_val, y_val, z_val])
    return np.array(points_list, dtype=np.float64)

```

```

def simulate_and_evaluate(theta_deg, uav_vel, drop_time, burst_delay,
sample_pts, return_segments=False):
    theta_rad = math.radians(theta_deg)
    direction_xy = np.array([math.cos(theta_rad), math.sin(theta_rad), 0.0],
dtype=np.float64)
    velocity_vec = uav_vel * direction_xy

    actual_drop_time = max(0.0, drop_time)
    actual_burst_time = actual_drop_time + max(0.0, burst_delay)

    uav_drop_pos = FY2_START_POS + velocity_vec * actual_drop_time

    horiz_displacement = velocity_vec * burst_delay
    vert_displacement = np.array([0.0, 0.0, -0.5 * GRAVITY_ACC * (burst_delay
** 2)], dtype=np.float64)
    burst_position = uav_drop_pos + horiz_displacement + vert_displacement

    start_time = actual_burst_time
    end_time = actual_burst_time + EFFECTIVE_DURATION

    time_points = np.arange(start_time, end_time + 1e-12, TIME_STEP)
    num_steps = len(time_points)

    cover_intervals = []
    is_covering = False
    segment_start = None

    missile_positions = M1_START_POS + missile_velocity_vec *
time_points.reshape(-1, 1)

    time_after_burst = time_points - actual_burst_time
    cloud_positions = np.empty((num_steps, 3), dtype=np.float64)
    cloud_positions[:, 0] = burst_position[0]
    cloud_positions[:, 1] = burst_position[1]
    cloud_positions[:, 2] = burst_position[2] - CLOUD_SINK_RATE *
time_after_burst

    coverage_status = np.zeros(num_steps, dtype=bool)
    for i in range(num_steps):
        coverage_status[i] = check_effectiveness_numba(
            missile_positions[i],
            cloud_positions[i],
            sample_pts,
            EFFECTIVE_RADIUS,
            CYLINDER_CENTER
        )

    for i in range(num_steps):
        if coverage_status[i] and not is_covering:
            is_covering = True
            segment_start = time_points[i]
        elif (not coverage_status[i]) and is_covering:
            is_covering = False
            cover_intervals.append((segment_start, time_points[i] -
TIME_STEP))
            segment_start = None

    if is_covering and segment_start is not None:
        cover_intervals.append((segment_start, end_time))

    total_coverage = 0.0
    for start, end in cover_intervals:
        total_coverage += (end - start)

    if return_segments:

```

```

        return total_coverage, cover_intervals, uav_drop_pos, burst_position,
actual_drop_time, actual_burst_time
    else:
        return total_coverage

param_bounds = {
    'theta_deg': (270.0, 280.0),
    'uav_vel': (70.0, 140.0),
    'drop_time': (0.0, 60.0),
    'burst_delay': (0.1, 8.0),
}

POPULATION_SIZE = 50
MAX_GENERATIONS = 300
ELITE_COUNT = 2
CROSSOVER_RATE = 0.8
MUTATION_RATE = 0.25
ETA_CROSSOVER = 10.0
ETA_MUTATION = 20.0
RANDOM_SEED = 42

bounds_array = [
    param_bounds['theta_deg'],
    param_bounds['uav_vel'],
    param_bounds['drop_time'],
    param_bounds['burst_delay'],
]

def initialize_population(size):
    population = []
    for _ in range(size):
        individual = np.array([
            np.random.uniform(*param_bounds['theta_deg']),
            np.random.uniform(*param_bounds['uav_vel']),
            np.random.uniform(*param_bounds['drop_time']),
            np.random.uniform(*param_bounds['burst_delay'])
        ], dtype=np.float64)
        population.append(individual)
    return np.array(population, dtype=np.float64)

def constrain_to_bounds(ind):
    ind[0] = np.clip(ind[0], *param_bounds['theta_deg'])
    ind[1] = np.clip(ind[1], *param_bounds['uav_vel'])
    ind[2] = np.clip(ind[2], *param_bounds['drop_time'])
    ind[3] = np.clip(ind[3], *param_bounds['burst_delay'])
    return ind

def select_by_tournament(fitness_values, k=2):
    idx1 = np.random.randint(0, len(fitness_values))
    idx2 = np.random.randint(0, len(fitness_values))
    return idx1 if fitness_values[idx1] >= fitness_values[idx2] else idx2

def perform_sbx_crossover(parent1, parent2, eta):
    if np.random.rand() > CROSSOVER_RATE:
        return parent1.copy(), parent2.copy()
    child1 = parent1.copy()
    child2 = parent2.copy()
    for i in range(len(parent1)):
        x1, x2 = parent1[i], parent2[i]
        if np.random.rand() < 0.5 and abs(x1 - x2) > 1e-14:
            x_low, x_high = min(x1, x2), max(x1, x2)
            u_val = np.random.rand()
            beta_val = 1.0 + (2.0 * (x_low - bounds_array[i][0]) / (x_high -
x_low))

            alpha_val = 2.0 - (beta_val ** -(eta + 1.0))
            if u_val <= 1.0 / alpha_val:

```

```

        beta_q = (u_val * alpha_val) ** (1.0 / (eta + 1.0))
    else:
        beta_q = (1.0 / (2.0 - u_val * alpha_val)) ** (1.0 / (eta +
1.0))

    child1[i] = 0.5 * ((x1 + x2) - beta_q * (x2 - x1))

    u_val = np.random.rand()
    beta_val = 1.0 + (2.0 * (bounds_array[i][1] - x_high) / (x_high -
x_low))

    alpha_val = 2.0 - (beta_val ** -(eta + 1.0))
    if u_val <= 1.0 / alpha_val:
        beta_q = (u_val * alpha_val) ** (1.0 / (eta + 1.0))
    else:
        beta_q = (1.0 / (2.0 - u_val * alpha_val)) ** (1.0 / (eta +
1.0))

    child2[i] = 0.5 * ((x1 + x2) + beta_q * (x2 - x1))
    else:
        child1[i] = x1
        child2[i] = x2
    return constrain_to_bounds(child1), constrain_to_bounds(child2)

def apply_polynomial_mutation(individual, eta):
    for i in range(len(individual)):
        if np.random.rand() < MUTATION_RATE:
            lower_bound, upper_bound = bounds_array[i]
            x_val = individual[i]
            if lower_bound == upper_bound:
                continue
            delta1 = (x_val - lower_bound) / (upper_bound - lower_bound)
            delta2 = (upper_bound - x_val) / (upper_bound - lower_bound)
            u_val = np.random.rand()
            mutation_power = 1.0 / (eta + 1.0)
            if u_val < 0.5:
                xy_val = 1.0 - delta1
                value = 2.0 * u_val + (1.0 - 2.0 * u_val) * (xy_val ** (eta +
1.0))

                delta_q = (value ** mutation_power) - 1.0
            else:
                xy_val = 1.0 - delta2
                value = 2.0 * (1.0 - u_val) + 2.0 * (u_val - 0.5) * (xy_val **
(eta + 1.0))

                delta_q = 1.0 - (value ** mutation_power)
            x_val = x_val + delta_q * (upper_bound - lower_bound)
            individual[i] = np.clip(x_val, lower_bound, upper_bound)
    return individual

def evaluate_fitness(population, sample_pts):
    fitness_scores = np.zeros(len(population), dtype=np.float64)
    for idx, ind in enumerate(population):
        theta_deg, uav_vel, drop_time, burst_delay = ind
        total_cover = simulate_and_evaluate(theta_deg, uav_vel, drop_time,
burst_delay, sample_pts)
        fitness_scores[idx] = total_cover**1.2
    return fitness_scores

def run_genetic_algorithm():
    samples = generate_cyl_samples(A_DEFAULT, B_DEFAULT)
    print(f"样本点数量: {len(samples)}")

    population = initialize_population(POPULATION_SIZE)
    fitness_scores = evaluate_fitness(population, samples)

    best_fitness_history = []
    best_individual_history = []

    best_index = int(np.argmax(fitness_scores))

```

```

best_fitness = float(fitness_scores[best_index])
best_individual = population[best_index].copy()
best_fitness_history.append(best_fitness)
best_individual_history.append(best_individual.copy())

stall_counter = 0
STALL_LIMIT = 50

start_time = time.time()

for gen in range(1, MAX_GENERATIONS + 1):
    new_population = []

    elites_indices = fitness_scores.argsort()[-ELITE_COUNT:][::-1]
    elites = [population[i].copy() for i in elites_indices]
    new_population.extend(elites)

    while len(new_population) < POPULATION_SIZE:
        idx1 = select_by_tournament(fitness_scores, k=2)
        idx2 = select_by_tournament(fitness_scores, k=2)
        parent1 = population[idx1].copy()
        parent2 = population[idx2].copy()

        child1, child2 = perform_sbx_crossover(parent1, parent2,
ETA_CROSSOVER)

        child1 = apply_polynomial_mutation(child1, ETA_MUTATION)
        child2 = apply_polynomial_mutation(child2, ETA_MUTATION)

        new_population.append(child1)
        if len(new_population) < POPULATION_SIZE:
            new_population.append(child2)

    population = np.array(new_population, dtype=np.float64)
    fitness_scores = evaluate_fitness(population, samples)

    current_index = int(np.argmax(fitness_scores))
    current_fitness = float(fitness_scores[current_index])
    current_best = population[current_index].copy()

    if current_fitness > best_fitness + 1e-9:
        best_fitness = current_fitness
        best_individual = current_best.copy()
        stall_counter = 0
    else:
        stall_counter += 1

    best_fitness_history.append(best_fitness)
    best_individual_history.append(best_individual.copy())

    if best_fitness != 0 and stall_counter >= STALL_LIMIT:
        print(f"[Early Stop] 连续 {STALL_LIMIT} 代未提升, 在第 {gen} 代停
止。")

        break

    if gen % 10 == 0 or gen == 1:
        elapsed = time.time() - start_time
        print(f"代 {gen:3d} | 代内最佳={current_fitness:.4f} | 历史最佳
={best_fitness:.4f} | 耗时: {elapsed:.2f}s")

    return best_individual, best_fitness, np.array(best_fitness_history),
samples

def main():
    print("=" * 60)
    print("FY2 无人机烟幕弹投放策略优化")
    print("=" * 60)

```

```

print(f"FY2 初始位置: {FY2_START_POS}")
print("开始遗传算法优化...")

best_ind, best_fit, best_curve, samples = run_genetic_algorithm()
theta_deg, uav_vel, t_drop, t_burst_delay = best_ind.tolist()

print("对最优解进行精细评估...")
total_cover, segments, drop_pos, burst_pos, drop_time, burst_time =
simulate_and_evaluate(
    theta_deg, uav_vel, t_drop, t_burst_delay, samples,
    return_segments=True
)

print("\n" + "=" * 50)
print("FY2 最优解结果")
print("=" * 50)
print(f"相对 x 轴正向的飞行倾角  $\theta$  (度) : {theta_deg:.4f}")
print(f"无人机巡航速度 v (m/s) : {uav_vel:.4f}")
print(f"投放延迟 t_drop (s) : {t_drop:.4f}")
print(f"起爆延迟  $\Delta t_{burst}$  (s) : {t_burst_delay:.4f}")

print("\n" + "-" * 50)
print("由最优解推得的位置")
print("-" * 50)
print(f"烟幕干扰弹投放点 (x, y, z) : ({drop_pos[0]:.3f},
{drop_pos[1]:.3f}, {drop_pos[2]:.3f})")
print(f"烟幕干扰弹起爆点 (x, y, z) : ({burst_pos[0]:.3f},
{burst_pos[1]:.3f}, {burst_pos[2]:.3f})")

print("\n" + "-" * 50)
print("遮蔽评估 (双判定)")
print("-" * 50)
print(f"最长'有效遮挡'总时长 (s) : {total_cover:.4f}")
if len(segments) == 0:
    print("无有效遮挡时间段。")
else:
    print("有效遮挡时间段: ")
    for k, (s, e) in enumerate(segments, 1):
        print(f" 段 {k:02d}: {s:.3f} s ~ {e:.3f} s (持续 {e - s:.3f}
s)")

plt.figure(figsize=(7.2, 4.8))
plt.plot(best_curve, lw=2, color='blue')
plt.xlabel("迭代代数")
plt.ylabel("历史最佳适应度(有效遮挡时长,s)")
plt.title("FY2 适应度随迭代次数的变化(历史最佳)")
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig("FY2_fitness_curve.png", dpi=160)
print("\n 已保存迭代曲线: FY2_fitness_curve.png")

if __name__ == "__main__":
    main()

```

附录七:

第五小问代码(部分)

```
import numpy as np
```

```

import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import differential_evolution
import warnings
import math
import time
from numba import jit

...
# 核心函数
@jit(nopython=True, cache=True)
def compute_segment_distance(P, A, B):
    """计算点 P 到线段 AB 的最短距离"""
    vec_AP = P - A
    vec_AB = B - A
    len_sq = np.dot(vec_AB, vec_AB)
    if len_sq <= 1e-24:
        return np.linalg.norm(P - A)
    t_param = np.dot(vec_AP, vec_AB) / len_sq
    if t_param <= 0.0:
        return np.linalg.norm(P - A)
    elif t_param >= 1.0:
        return np.linalg.norm(P - B)
    else:
        proj_point = A + t_param * vec_AB
        return np.linalg.norm(P - proj_point)

@jit(nopython=True, cache=True)
def check_coverage_batch(missile_positions, smoke_positions, sample_points,
num_smokes):
    """批量检查遮蔽状态"""
    time_steps = len(missile_positions)
    num_samples = len(sample_points)
    coverage_result = np.zeros(time_steps, dtype=np.bool_)

    for t_index in range(time_steps):
        current_missile_pos = missile_positions[t_index]
        all_covered = True

        for s_index in range(num_samples):
            sample_pt = sample_points[s_index]
            covered = False

            for smoke_index in range(num_smokes):
                smoke_pos = smoke_positions[smoke_index, t_index]
                dist = compute_segment_distance(smoke_pos,
current_missile_pos, sample_pt)
                if dist < SMOKE_RADIUS:
                    covered = True
                    break

            if not covered:
                all_covered = False
                break

        coverage_result[t_index] = all_covered

    return coverage_result

def generate_cylinder_samples(angle_samples, height_samples):
    """生成圆柱体表面采样点"""
    points = []
    radius = TARGET_CYLINDER["radius"]
    height = TARGET_CYLINDER["height"]
    center = TARGET_CYLINDER["center_pos"]

```

```

for layer in range(height_samples + 1):
    z_val = center[2] + layer * height / height_samples
    for i in range(angle_samples):
        angle = 2.0 * math.pi * i / angle_samples
        x_val = center[0] + radius * math.cos(angle)
        y_val = center[1] + radius * math.sin(angle)
        points.append([x_val, y_val, z_val])

for z_offset in [0, height]:
    z_val = center[2] + z_offset
    points.append([center[0], center[1], z_val])
    for i in range(0, angle_samples, max(1, angle_samples // 4)):
        angle = 2.0 * math.pi * i / angle_samples
        x_val = center[0] + radius * math.cos(angle)
        y_val = center[1] + radius * math.sin(angle)
        points.append([x_val, y_val, z_val])

return np.array(points, dtype=np.float64)

def initialize_system():
    """初始化系统参数"""
    TARGET_CYLINDER["sample_pts"] = generate_cylinder_samples(CIRCLE_SAMPLES,
HEIGHT_SAMPLES)

    for m_name, m_info in MISSILE_DATA.items():
        start_pos = m_info["start_pos"]
        dir_vector = -start_pos / np.linalg.norm(start_pos)
        m_info["direction"] = dir_vector * MISSILE_VELOCITY
        m_info["flight_duration"] = np.linalg.norm(start_pos) /
MISSILE_VELOCITY

def calculate_missile_position(missile_id, time_val):
    """计算导弹在指定时间的位置"""
    m_info = MISSILE_DATA[missile_id]
    if time_val > m_info["flight_duration"]:
        return m_info["start_pos"] + m_info["direction"] *
m_info["flight_duration"]
    return m_info["start_pos"] + m_info["direction"] * time_val

def calculate_uav_position(uav_id, time_val):
    """计算无人机在指定时间的位置"""
    uav = UAV_FLEET[uav_id]
    if uav["velocity"] is None or uav["heading"] is None:
        return uav["start_pos"]

    velocity_vec = np.array([uav["velocity"] * np.cos(uav["heading"]),
uav["velocity"] * np.sin(uav["heading"]), 0])
    return uav["start_pos"] + velocity_vec * time_val

def calculate_smoke_position(uav_id, drop_time, detonation_delay, time_val):
    """计算烟雾弹在指定时间的位置"""
    uav = UAV_FLEET[uav_id]
    if time_val < drop_time:
        return None

    drop_position = calculate_uav_position(uav_id, drop_time)

    if time_val < drop_time + detonation_delay:
        delta_t = time_val - drop_time
        velocity_vec = np.array([uav["velocity"] * np.cos(uav["heading"]),
uav["velocity"] * np.sin(uav["heading"]), 0])
        x_pos = drop_position[0] + velocity_vec[0] * delta_t
        y_pos = drop_position[1] + velocity_vec[1] * delta_t
        z_pos = drop_position[2] - 0.5 * GRAVITY_CONST * delta_t ** 2

```



```

        return np.array([x_pos, y_pos, max(z_pos, 0.1)])

    detonation_time = drop_time + detonation_delay
    if time_val > detonation_time + SMOKE_DURATION:
        return None

    delta_t_det = detonation_delay
    velocity_vec = np.array([uav["velocity"] * np.cos(uav["heading"]),
                             uav["velocity"] * np.sin(uav["heading"]), 0])
    det_x = drop_position[0] + velocity_vec[0] * delta_t_det
    det_y = drop_position[1] + velocity_vec[1] * delta_t_det
    det_z = drop_position[2] - 0.5 * GRAVITY_CONST * delta_t_det ** 2

    if det_z < 0:
        det_z = 0.1

    delta_t_after = time_val - detonation_time
    z_pos = det_z - SMOKE_DESCENT_RATE * delta_t_after
    return np.array([det_x, det_y, max(z_pos, 0.1)])

# 智能任务分配系统
def assign_tasks_intelligently():
    """智能任务分配系统"""
    uav_names = list(UAV_FLEET.keys())
    missile_names = list(MISSILE_DATA.keys())

    affinity_matrix = np.zeros((len(uav_names), len(missile_names)))

    for i, uav_id in enumerate(uav_names):
        uav_pos = UAV_FLEET[uav_id]["start_pos"]
        uav_avg_speed = np.mean(UAV_FLEET[uav_id]["vel_range"])

        for j, missile_id in enumerate(missile_names):
            missile_pos = MISSILE_DATA[missile_id]["start_pos"]
            missile_time = MISSILE_DATA[missile_id]["flight_duration"]

            distance = np.linalg.norm(uav_pos - missile_pos)
            distance_factor = 1.0 / (1.0 + distance / 20000)

            time_factor = min(missile_time / 50.0, 2.0)

            speed_diff = abs(uav_avg_speed - MISSILE_VELOCITY)
            speed_factor = 1.0 / (1.0 + speed_diff / 250)

            height_diff = abs(uav_pos[2] - missile_pos[2])
            height_factor = 1.0 / (1.0 + height_diff / 2000)

            affinity_matrix[i][j] = distance_factor * time_factor *
speed_factor * height_factor

    assignments = {}

    for i, uav_id in enumerate(uav_names):
        missile_scores = [(missile_names[j], affinity_matrix[i][j]) for j in
range(len(missile_names))]
        missile_scores.sort(key=lambda x: x[1], reverse=True)
        assignments[uav_id] = [m for m, _ in missile_scores[:2]]

    missile_coverage = {m: [] for m in missile_names}
    for uav_id, missiles in assignments.items():
        for missile_id in missiles:
            missile_coverage[missile_id].append(uav_id)

    for missile_id, uavs in missile_coverage.items():
        if len(uavs) < 2:

```

```

        unassigned_uavs = [u for u in uav_names if missile_id not in
assignments[u]]
        if unassigned_uavs:
            best_uav = max(unassigned_uavs,
                            key=lambda u:
affinity_matrix[uav_names.index(u)][missile_names.index(missile_id)])
            assignments[best_uav].append(missile_id)

        for uav_id, missiles in assignments.items():
            UAV_FLEET[uav_id]["target_missiles"] = list(set(missiles))

    print(f"任务分配结果:")
    for uav_id, missiles in assignments.items():
        print(f" {uav_id}: {missiles}")

    return assignments

# 无人机轨迹优化
def optimize_uav_trajectory(uav_id, target_missiles, max_attempts=3):
    """优化无人机轨迹"""
    uav = UAV_FLEET[uav_id]
    v_min, v_max = uav["vel_range"]
    max_smokes = uav["max_smokes"]

    best_solution = {
        "smokes": [],
        "total_coverage": 0.0,
        "velocity": None,
        "heading": None
    }

    print(f"\n[{uav_id}] 开始优化, 目标导弹: {target_missiles}")

    for attempt in range(max_attempts):
        for primary_missile in target_missiles:
            current_solution = optimize_for_missile(
                uav_id, primary_missile, target_missiles, attempt
            )

            if current_solution and current_solution["total_coverage"] >
best_solution["total_coverage"]:
                best_solution = current_solution
                print(f"[{uav_id}] 找到更好解:
{current_solution['total_coverage']:.4f}s")

            if best_solution["smokes"]:
                uav["smoke_list"] = best_solution["smokes"]
                uav["velocity"] = best_solution["velocity"]
                uav["heading"] = best_solution["heading"]

                print(f"[{uav_id}] 优化成功: {len(best_solution['smokes'])}枚烟幕弹, "
                    f"总遮蔽{best_solution['total_coverage']:.4f}s")
                return best_solution["smokes"]

    print(f"[{uav_id}] 优化失败")
    return []

def optimize_for_missile(uav_id, primary_missile, all_targets, seed=0):
    """针对特定导弹进行优化"""
    uav = UAV_FLEET[uav_id]
    v_min, v_max = uav["vel_range"]
    max_smokes = uav["max_smokes"]

    np.random.seed(seed * 42 + hash(uav_id + primary_missile) % 1000)

```

```

        max_flight_time = min([MISSILE_DATA[m]["flight_duration"] for m in
all_targets])

    best_solution = {"smokes": [], "total_coverage": 0.0, "velocity": None,
"heading": None}

    for num_smokes in range(max_smokes, 0, -1):
        print(f" [{uav_id}] 尝试 {num_smokes} 枚烟幕弹对 {primary_missile}")

        bounds = [
            (v_min, v_max),
            (0, 2 * np.pi),
        ]

        for i in range(num_smokes):
            bounds.append((0.5, max_flight_time - 2.0))
            bounds.append((0.5, 8.0))

        def objective(x):
            v = x[0]
            theta = x[1]

            smoke_params = []
            for i in range(num_smokes):
                drop_time = x[2 + 2*i]
                det_delay = x[2 + 2*i + 1]
                smoke_params.append({"drop_time": drop_time, "det_delay":
det_delay})

            drop_times = [sp["drop_time"] for sp in smoke_params]
            drop_times.sort()
            for j in range(1, len(drop_times)):
                if drop_times[j] - drop_times[j-1] < DROP_GAP:
                    return 1e6

            primary_coverage = calculate_coverage(
                uav_id, primary_missile, v, theta, smoke_params
            )

            secondary_coverage = 0.0
            for other_missile in all_targets:
                if other_missile != primary_missile:
                    secondary_coverage += 0.3 * calculate_coverage(
                        uav_id, other_missile, v, theta, smoke_params
                    )

            total_coverage = primary_coverage + secondary_coverage
            return -total_coverage

        try:
            result = differential_evolution(
                func=objective,
                bounds=bounds,
                mutation=OPT_PARAMS['mutation_range'],
                recombination=OPT_PARAMS['crossover_rate'],
                popsize=OPT_PARAMS['population_size'],
                maxiter=OPT_PARAMS['max_iterations'],
                tol=OPT_PARAMS['tolerance'],
                disp=False,
                polish=True,
                workers=1
            )

            if result.success or -result.fun > 0.1:
                v_opt = result.x[0]
                theta_opt = result.x[1]

```

```

        smoke_params = []
        for i in range(num_smokes):
            drop_time = result.x[2 + 2*i]
            det_delay = result.x[2 + 2*i + 1]
            smoke_params.append({"drop_time": drop_time, "det_delay":
det_delay})

        total_coverage = calculate_coverage(
            uav_id, primary_missile, v_opt, theta_opt, smoke_params
        )

        if total_coverage > best_solution["total_coverage"]:
            smokes = []
            for i, sp in enumerate(smoke_params):
                det_time = sp["drop_time"] + sp["det_delay"]

                original_vel = uav["velocity"]
                original_head = uav["heading"]
                uav["velocity"] = v_opt
                uav["heading"] = theta_opt

                det_pos = calculate_smoke_position(uav_id,
sp["drop_time"], sp["det_delay"], det_time)

                uav["velocity"] = original_vel
                uav["heading"] = original_head

                smokes.append({
                    "v": v_opt,
                    "theta": theta_opt,
                    "drop_time": sp["drop_time"],
                    "det_delay": sp["det_delay"],
                    "det_time": det_time,
                    "det_pos": det_pos,
                    "effective_time": total_coverage / num_smokes,
                    "missile": primary_missile
                })

            best_solution = {
                "smokes": smokes,
                "total_coverage": total_coverage,
                "velocity": v_opt,
                "heading": theta_opt
            }

        print(f"    找到 {num_smokes} 枚烟幕弹解，遮蔽时长：
{total_coverage:.4f}s")

        if total_coverage > 2.0:
            break

    except Exception as e:
        print(f"    优化异常：{e}")
        continue

    return best_solution if best_solution["smokes"] else None

def calculate_coverage(uav_id, missile_id, velocity, heading, smoke_params):
    """计算遮蔽时长"""
    if not smoke_params:
        return 0.0

    uav = UAV_FLEET[uav_id]
    original_vel = uav["velocity"]
    original_head = uav["heading"]

```

```

uav["velocity"] = velocity
uav["heading"] = heading

try:
    if not (uav["vel_range"][0] <= velocity <= uav["vel_range"][1]):
        return 0.0

    drop_times = [sp["drop_time"] for sp in smoke_params]
    drop_times.sort()
    for i in range(1, len(drop_times)):
        if drop_times[i] - drop_times[i-1] < DROP_GAP:
            return 0.0

    burst_positions = []
    burst_times = []

    for sp in smoke_params:
        drop_time = sp["drop_time"]
        det_delay = sp["det_delay"]

        drop_pos = calculate_uav_position(uav_id, drop_time)
        det_z = drop_pos[2] - 0.5 * GRAVITY_CONST * det_delay ** 2
        if det_z < 0:
            return 0.0

        velocity_vec = np.array([velocity * np.cos(heading), velocity *
np.sin(heading), 0])
        det_x = drop_pos[0] + velocity_vec[0] * det_delay
        det_y = drop_pos[1] + velocity_vec[1] * det_delay
        det_z = max(det_z, 0.1)

        burst_positions.append(np.array([det_x, det_y, det_z],
dtype=np.float64))
        burst_times.append(drop_time + det_delay)

    earliest = min(burst_times)
    latest = max(burst_times) + SMOKE_DURATION
    latest = min(latest, MISSILE_DATA[missile_id]["flight_duration"] + 1)

    if earliest >= latest - 1e-6:
        return 0.0

    time_points = np.arange(earliest, latest, TIME_INCREMENT)
    if len(time_points) == 0:
        return 0.0

    missile_positions = np.array([calculate_missile_position(missile_id,
t) for t in time_points])

    smoke_trajectories = compute_smoke_trajectories(
        np.array(burst_positions), np.array(burst_times), time_points
    )

    num_smokes = len(burst_positions)
    coverage_status = check_coverage_batch(
        missile_positions, smoke_trajectories,
TARGET_CYLINDER["sample_pts"], num_smokes
    )

    effective_duration = np.sum(coverage_status) * TIME_INCREMENT
    return effective_duration

finally:
    uav["velocity"] = original_vel
    uav["heading"] = original_head

```

```

@jit(nopython=True, cache=True)
def compute_smoke_trajectories(burst_positions, burst_times, time_points):
    """计算烟幕弹轨迹"""
    num_smokes = len(burst_positions)
    num_times = len(time_points)
    smoke_positions = np.full((num_smokes, num_times, 3), -1000.0,
dtype=np.float64)

    for i in range(num_smokes):
        burst_pos = burst_positions[i]
        burst_time = burst_times[i]

        for j in range(num_times):
            time_after_burst = time_points[j] - burst_time
            if 0.0 <= time_after_burst <= SMOKE_DURATION:
                smoke_positions[i, j, 0] = burst_pos[0]
                smoke_positions[i, j, 1] = burst_pos[1]
                smoke_positions[i, j, 2] = burst_pos[2] - SMOKE_DESCENT_RATE *
time_after_burst

    return smoke_positions

# 全局协调优化
def coordinate_globally():
    """全局协调优化"""
    print("="*60)
    print("全局协调优化系统")
    print("="*60)

    for uav_id in UAV_FLEET:
        UAV_FLEET[uav_id]["smoke_list"] = []
        UAV_FLEET[uav_id]["velocity"] = None
        UAV_FLEET[uav_id]["heading"] = None
        UAV_FLEET[uav_id]["target_missiles"] = []

    assignments = assign_tasks_intelligently()

    all_smokes = []
    successful_uavs = []

    for uav_id, target_missiles in assignments.items():
        if target_missiles:
            print(f"\n 正在优化 {uav_id}...")
            smokes = optimize_uav_trajectory(uav_id, target_missiles)

            if smokes:
                uav_smokes = [{**smoke, "uav": uav_id} for smoke in smokes]
                all_smokes.extend(uav_smokes)
                successful_uavs.append(uav_id)

    covered_missiles = set([smoke["missile"] for smoke in all_smokes])
    uncovered_missiles = set(MISSILE_DATA.keys()) - covered_missiles
    unused_uavs = set(UAV_FLEET.keys()) - set(successful_uavs)

    print(f"\n 优化结果统计:")
    print(f"成功优化的无人机: {successful_uavs}")
    print(f"未使用的无人机: {list(unused_uavs)}")
    print(f"覆盖的导弹: {list(covered_missiles)}")
    print(f"未覆盖的导弹: {list(uncovered_missiles)}")

    return all_smokes
...

```

--